

**PINYA-PIC: A Machine Learning-Driven Mobile Application for Mealybug
Infestation Detection with Integrated Geotagging and Geofencing Features**



Undergraduate Capstone Project Presented to the Faculty of the IT & Physics
Department College of Natural Sciences and Mathematics Mindanao State University
– General Santos City

In Partial Fulfillment of the Requirements for the Degree Bachelor of Science in
Information Technology

By:

GENAMAE O. BACAY

JANRAFAEL F. MORGA

ANGELEI P. MORILLO

KHALIL ANTON AMBALONG, MS

Adviser

December 2024

ABSTRACT

PINYA-PIC is a machine learning-driven mobile application designed to address the challenges pineapple farmers in Polomolok, South Cotabato face due to mealybug infestations. Mealybugs (*Dysmicoccus brevipes*) harm crops by reducing photosynthesis and spreading diseases like mealybug wilt, causing significant economic losses. Current pest management relies on inefficient manual inspections that are time-consuming, labor-intensive, and often inadequate to address the rapid spread of infestations.

This study developed PINYA-PIC as an Android-based, machine learning-driven mobile application for assisting pineapple farmers in documenting and reviewing mealybug infestations in Polomolok, South Cotabato. The application combines on-device image analysis, geotagging, local field-boundary checking, saved-image history, and cloud synchronization to support field monitoring in environments where internet connectivity may be unreliable.

The developed application follows a local-first architecture. Captured results are stored in SQLite before cloud upload, while Supabase provides authentication, field metadata, detection storage, and account-linked restoration for records that have already synchronized. Major implemented features include field creation and editing, saved-images gallery, 7-day dashboard analytics, detections map with severity-based visualization, device-unlock protection, English/Filipino language switching, online-required prompts for internet-dependent actions, and feedback submission through either email or an in-app web form. Geofencing in the system is implemented as local point-in-polygon validation of field boundaries rather than operating-system background geofence triggers.

The detection workflow is still-image based rather than a live camera stream. Users capture a photo or choose one from the gallery, after which a bundled TensorFlow Lite model based on the Ultralytics YOLO26n architecture is executed on-device with EXIF-aware preprocessing, coordinate remapping, and Dart-side Non-Maximum Suppression. Earlier stages of the project also documented YOLO11-based experimentation, while later development referenced a newer YOLO26n training pipeline. Accordingly, the deployed system is described in this manuscript as using a YOLO26n (Ultralytics YOLO26 Nano) TensorFlow Lite model. PINYA-PIC contributes to Sustainable Development Goals 2 (Zero Hunger) and 12 (Responsible Consumption and Production) by supporting earlier pest documentation, more targeted interventions, and better spatial awareness of infestation patterns.

Keywords: pineapple farming, mealybug infestation, machine learning, YOLO26n, YOLO26, TensorFlow Lite object detection, geotagging, precision agriculture, local-first architecture, Supabase, TensorFlow Lite, System Usability Scale

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS.....	4
LIST OF FIGURES	7
CHAPTER III	7
APPENDIX A.....	9
LIST OF TABLES	10
CHAPTER I.....	11
INTRODUCTION	11
1.0 Background of the Study	11
1.1 Objectives of the Study	13
1.1.1 General Objective	13
1.1.2 Specific Objectives.....	13
1.2 Significance of the Study	14
1.2.1 Farmers	14
1.2.2 Agricultural Technicians	15
1.2.3 Researchers.....	15
1.2.4 Department of Agriculture	15
1.2.5 Contribution to Sustainable Development Goals	16
1.3 Scope and Limitation	16
1.4 Definition of Terms.....	19
CHAPTER II	23
REVIEW OF RELATED LITERATURE AND TECHNOLOGIES	23
2.0 Related Literature	23
2.0.1 Machine Learning in Pest Detection	23
2.0.2 Deep Learning Models for Object Detection	24
2.0.3 YOLO Architecture Evolution	25
2.0.4 Geospatial Technologies in Agriculture	26
2.0.5 Offline-First Mobile Architectures	26
2.0.6 Mobile Usability in Agricultural Contexts	27
2.0.7 Synthesis of Related Literature	27

2.1 Related Technologies.....	28
2.1.1 Agrio	28
2.1.2 Cropx.....	29
2.1.3 Plantix.....	29
2.1.4 SPIDTECH	30
2.1.5 Synthesis of Related Technologies	30
CHAPTER III	33
METHODOLOGY.....	33
3.0 Modified Waterfall Software Development Methodology	33
3.1 Requirements Gathering	34
3.2 Design.....	35
3.2.1 System Design Specification.....	35
3.2.2 Conceptual Framework	36
3.2.3 Operational Framework.....	37
3.2.4 Defined Users and User Roles.....	38
3.2.5 Defined Functionalities and Processes	39
3.2.6 Use Case Diagram.....	39
3.2.7 Activity Diagrams	41
3.2.8 Entity-Relationship Diagram	41
3.2.9 High-Fidelity Prototypes	42
3.2.10 System Architecture	60
3.3 Implementation	62
3.3.1 Technology Background	62
3.3.2 Software Stack.....	62
3.3.3 System Development Timeline.....	66
3.3.4 Key Implementation Components.....	67
3.4 Testing	69
3.4.1 Qualifying Parameters for Usability Testing.....	69
3.4.2 System Usability Scale Testing Framework.....	70
3.4.3 Usability Testing Method.....	70
3.4.4 Functional Testing.....	71
3.4.5 Performance Testing	72

3.4.6 Offline Mode Testing.....	72
3.4.7 Security Testing.....	72
3.5 Deployment.....	73
3.5.1 Deployment Diagram.....	73
3.5.2 Build Configuration	74
3.5.3 Release APK Generation	74
3.5.4 Team Sharing Methodology.....	75
CHAPTER IV	76
RESULTS AND DISCUSSION	76
4.0 System Features and Functionalities	76
4.0.1 Authentication and Onboarding	76
4.0.2 Dashboard and Navigation	77
4.0.3 Field and Plot Management	77
4.0.4 Camera and Detection Pipeline	78
4.0.5 Geospatial Services.....	78
4.0.6 Offline Synchronization	79
4.0.7 Analytics and Visualization	80
4.0.8 Feedback System	80
4.0.9 Profile Management	81
4.0.10 Network Awareness.....	81
4.0.11 Saved Images Gallery and Account-Linked Restoration.....	81
4.1 Model Performance Evaluation.....	82
4.1.1 Documented Training Metrics	82
4.1.2 Inference Performance	83
4.1.3 Model Size Optimization	84
4.2 Usability Evaluation Results	84
4.2.1 Participant Demographics	84
4.2.2 SUS Scores for Valid Respondents (n = 10)	86
4.2.3 Per-Question Analysis	88
4.2.4 Qualitative Feedback Analysis	90
4.2.5 Task Completion Analysis.....	92
4.2.6 Correlation Analysis	94

4.3 Real-World Deployment Statistics	95
4.3.1 Overall Detection Metrics	95
4.3.2 Confidence Score Distribution	97
4.3.3 Geotagging and Spatial Consistency	97
4.3.4 Synchronization and Offline Operation	97
4.3.5 Comparison with Validation Metrics.....	98
4.4 Summary of Results	98
CHAPTER V	100
CONCLUSION AND RECOMMENDATIONS	100
5.0 Conclusion	100
5.0.1 Summary of Contributions	100
5.0.2 Achievement of Objectives	103
5.0.3 Technical Contributions	103
5.0.4 Usability Findings	104
5.1 Recommendations	106
5.1.1 Immediate Improvements.....	106
5.1.2 Future Enhancements	107
5.1.3 Research Directions	108
APPENDIX B.....	109
SYSTEM DIAGRAMS	109
REFERENCES	122

LIST OF FIGURES

CHAPTER III

- Figure 3.0 Modified Waterfall Software Development Methodology
- Figure 3.1 Expected Methodology Outputs
- Figure 3.2.2 Conceptual Framework
- Figure 3.2.3 Operational Framework

- Figure 3.2.6 Use Case Diagram
- Figure 3.2.8 Entity-Relationship Diagram
- Figure 3.2.9.1 Sign Up Page
- Figure 3.2.9.2 Login Page
- Figure 3.2.9.3 Field Input Page
- Figure 3.2.9.4 Home Page
- Figure 3.2.9.5 Diagnosis Page
- Figure 3.2.9.6 My Fields Page
- Figure 3.2.9.6.1 Field Details Page
- Figure 3.2.9.6.2.1 Captured Photos Details Page
- Figure 3.2.9.7 More Page
- Figure 3.2.9.8 Settings Page
- Figure 3.2.9.9 Analysis Page
- Figure 3.2.9.10 Field Map Page
- Figure 3.2.9.10.1 Field Boundary Editor Page
- Figure 3.2.9.10.2 Detections Map Page
- Figure 3.3.1.7 Gantt Chart
- Figure 3.5 Deployment Diagram

- Figure 4.6 Diagnose Tab: 7-Day Infestation Trend
- Figure 4.7 Confidence Score Distribution Histogram

APPENDIX A

- Figure A1: Create User Account
 - Figure A2: Edit User Account
 - Figure A3: Log In
 - Figure A4: Create New Field
 - Figure A5: Select Field
 - Figure A6: View Field Details
 - Figure A7: View Mealybug Infestation Count and Percentage
 - Figure A8: Delete Image
 - Figure A9: Capture Image
 - Figure A10: View Map
 - Figure A11: Add Geolocation Form
 - Figure A12: Remove Pineapple Field Geotag
 - Figure A13: Update Pineapple Field Boundary
-

LIST OF TABLES

Table 2.1. Comparison of Existing Applications and Proposed Application

Table 3.1. Defined Users and Their Roles

Table 3.2. Defined Functionalities and Processes

Table 4.1. Archived YOLO11 Training Metrics (Pre-Deployment Model)

Table 4.2. Inference Time by Device

Table 4.3. Model Size After Quantization

Table 4.4. Participant Demographics

Table 4.5. SUS Scores for Valid Respondents

Table 4.6. SUS Score Distribution

Table 4.7. Mean Scores per SUS Item

Table 4.8. Task Completion Times

Table 4.9. Correlation with SUS Score

Table 4.10. Summary of Field Detections

CHAPTER I

INTRODUCTION

1.0 Background of the Study

Pineapple farming is a vital economic activity in tropical and subtropical regions, providing nutritional benefits and economic opportunities. The growing demand for pineapple consumption and export underscores its significance (Hossain, M. F., 2015). The Philippines is the third-largest pineapple producer globally, with major production centers in Mindanao, particularly in Bukidnon, South Cotabato, and Davao del Norte. In South Cotabato, Polomolok serves as the heart of pineapple production, hosting large-scale plantations and thousands of smallholder farmers who depend on pineapple cultivation for their livelihoods.

However, this industry faces persistent challenges from mealybug wilt, a destructive disease caused by a parasitic complex involving Pineapple Mealybug Wilt-associated Viruses (PMWaVs) and mealybugs (*Dysmicoccus brevipes*), which act as vectors for these viruses (N'Guessan, L., et al. 2024). Adult pineapple mealybugs are small, approximately 1 mm wide, with a fuzzy appearance due to white wax and a pinkish hue underneath. They feed on the sap of pineapple plants, reducing photosynthesis and growth, and causing stunted growth, poor fruit quality, and, in severe cases, plant death. Their feeding also results in honeydew excretion, promoting sooty mold development and virus transmission (Canario, D. V. P., et al. 2017).

In Polomolok, South Cotabato, mealybug wilt disease presents a significant challenge for pineapple plantations. Local farmers, during interviews in Landan, reported that infestations not only lower crop productivity but also increase pest

management costs. Current practices depend heavily on manual inspection, which is time-consuming, labor-intensive, and often inadequate to address the rapid spread of infestations. Farmers typically conduct visual inspections of their fields, searching for visible signs of mealybugs or the characteristic symptoms of wilt disease. This approach has inherent limitations: time consumption, delayed detection, inaccurate assessment, and inefficient pesticide application.

Advancements in machine learning offer innovative solutions to pest management in agriculture. By applying computer vision, machine learning models such as YOLO can analyze crop images to detect pest infestations with high accuracy. When integrated with geotagging and geofencing, these technologies provide precise data on infestation locations and high-risk areas, enabling farmers to implement targeted and efficient pest control measures. This approach aligns with precision agriculture goals, optimizing resource use and promoting sustainable farming practices (Bendre et al., 2021).

To address these challenges, PINYA-PIC, a machine learning-driven mobile application, has been developed to detect mealybug infestations in pineapple plantations. By integrating machine learning, geotagging, and geofencing, PINYA-PIC provides a cost-effective and user-friendly solution for farmers. It identifies mealybug presence, assesses infestation density, and provides actionable insights. PINYA-PIC contributes primarily to Sustainable Development Goals 2 (Zero Hunger) and 12 (Responsible Consumption and Production), with additional support for SDG 9 (Industry, Innovation, and Infrastructure) and SDG 15 (Life on Land), by automating pest detection and delivering spatial data to enhance infestation management. This

helps reduce economic losses, promote sustainable farming practices, and support the long-term productivity of pineapple farming.

1.1 Objectives of the Study

1.1.1 General Objective

The objective of this research is to develop a mobile application that integrates AI-driven mealybug detection with geotagging and geofencing capabilities to enhance pest management in pineapple plantations. The system aims to provide insights into mealybug presence and population density, support targeted pest control efforts, and visualize historical data, in order to improve the sustainability and productivity of pineapple farming in Polomolok, South Cotabato.

1.1.2 Specific Objectives

1. Develop and integrate a compact YOLO26n-based TensorFlow Lite model suitable for on-device mealybug detection, with documented performance indicators such as precision, recall, mAP, and practical inference time.
2. Develop an Android mobile application with a local-first architecture that combines SQLite persistence and Supabase synchronization for field records, detections, and saved-image history.
3. Implement field management capabilities that allow users to create, edit, and maintain field names, preview images, and boundary information without recreating records.
4. Create dashboard and map visualizations that summarize recent detections through a 7-day trend chart, infestation-rate statistics, and severity-based spatial markers.

5. Implement robust location and mapping support, including current-position acquisition, last-known fallback when necessary, EXIF GPS reading, manual map-based pinning, and local point-in-polygon boundary checking.
6. Develop resilient user-support features including email feedback fallback, an in-app feedback form, network-aware prompts for online-only actions, and clear messaging when connectivity or GPS is unavailable.
7. Enhance the application with device unlock, saved-images gallery, account-linked restoration for uploaded records, Filipino language support, coordinate transformation for accurate overlays, and split-per-ABI build support.
8. Evaluate the usability and practical utility of the application with target users using the System Usability Scale (SUS) and task-based testing.

1.2 Significance of the Study

1.2.1 Farmers

The development of the PINYA-PIC application provides pineapple farmers with an innovative tool to efficiently detect mealybug infestations in their plantations. By integrating AI-powered detection with geotagging and geofencing features, farmers can pinpoint affected areas and monitor the population density of mealybugs. This allows for targeted interventions, reducing the need for widespread pesticide use, lowering operational costs, and improving crop health. The historical data and insights visualized through the mobile application enable farmers to make informed decisions, enhancing their pest management practices and leading to healthier, more productive plantations.

1.2.2 Agricultural Technicians

For agricultural technicians and extension workers, PINYA-PIC offers a standardized tool for assessing pest infestations across multiple farms. The consistent detection methodology reduces variability in assessments and provides objective data for decision-making. The geotagging capabilities enable precise documentation of infestation locations, facilitating targeted extension recommendations and more effective support for farmers.

1.2.3 Researchers

For researchers, PINYA-PIC offers an opportunity to explore the integration of AI in pest detection and farm management. The system's ability to track mealybug population density through advanced image recognition algorithms provides valuable data that can be used for further studies on pest control methods and agricultural sustainability. The offline synchronization architecture demonstrates a viable approach for deploying AI applications in resource-constrained environments, contributing to the body of knowledge on mobile AI deployment in developing regions.

1.2.4 Department of Agriculture

PINYA-PIC aligns with the Department of Agriculture's goals of improving agricultural productivity, sustainability, and pest management in the Philippines. By providing farmers with an easy-to-use, data-driven tool, the system supports the DA's mission to enhance the efficiency of local farming practices and reduce crop losses caused by pests. The geotagging and geofencing features also contribute to more effective monitoring of pest outbreaks at a regional level, enabling the DA to implement targeted interventions and programs.

1.2.5 Contribution to Sustainable Development Goals

PINYA-PIC directly contributes to multiple Sustainable Development Goals:

- SDG 2: Zero Hunger – Improves agricultural productivity by enabling early detection of pest infestations, reduces crop losses that would otherwise contribute to food insecurity, and supports sustainable food production systems.
- SDG 12: Responsible Consumption and Production – Reduces pesticide use through targeted interventions, promotes efficient resource utilization in agriculture, and minimizes environmental impact of pest management.
- SDG 9: Industry, Innovation, and Infrastructure – Applies machine learning innovation to agricultural challenges and demonstrates technological infrastructure for smallholder farmers.
- SDG 15: Life on Land – Reduces chemical runoff from excessive pesticide application and supports sustainable agricultural practices that preserve ecosystems.

1.3 Scope and Limitation

This study focuses on the design, implementation, and evaluation of a mobile application for documenting and reviewing mealybug infestations in pineapple farms. Its scope covers four major areas: on-device image inference, Android mobile application development, local-first data management with cloud synchronization, and geospatial visualization of saved detections.

For model integration, the mobile application uses a bundled TensorFlow Lite detector from the YOLO26n and performs image preprocessing, EXIF orientation

correction, coordinate remapping, confidence filtering, and Dart-side Non-Maximum Suppression on-device. Earlier development stages explored YOLO11 pipelines; however, the deployed system uses a YOLO26n model exported to TensorFlow Lite.

For mobile application development, the implemented system is field-focused rather than plot-focused. It supports account creation and sign-in, onboarding, field creation and editing, boundary mapping, captured-image review, dashboard analytics, detections map visualization, settings, profile management, feedback submission, saved-images gallery, and account-linked restoration of uploaded detections. The application is implemented in Flutter for Android and uses SQLite for local persistence with Supabase for cloud synchronization, authentication, and storage.

For local data handling, SQLite stores land boundaries, captured photos, upload queue entries, cached field metadata, and a legacy local detection table. New detections are written locally first, then uploaded to Supabase when connectivity and authentication are available. Restoration after reinstall is limited to detections that were previously uploaded; restored records may rely on remote image URLs rather than restored local files.

For geospatial support, the application records coordinates from current GPS, last-known GPS when necessary, EXIF metadata when available, or user-selected map pins. Geofencing is implemented as local point-in-polygon validation against stored field boundaries. The application also includes network-aware dialogs, device unlock, English/Filipino language switching, and a severity-based map view for uploaded detections.

Usability evaluation in this manuscript is based on the recorded SUS assessment and qualitative feedback gathered from farmers in Polomolok, South Cotabato.

Despite the comprehensive scope, the study acknowledges several limitations:

- **Geographic scope:** The research is confined to pineapple plantations in Polomolok, South Cotabato. Although the application may be adaptable to other regions, the usability evaluation and feature validation were conducted exclusively with farmers from this area; variations in mealybug populations, farming practices, and technology infrastructure elsewhere may affect generalizability.
- **Dataset dependency:** The model's detection accuracy relies on the quality and diversity of the Roboflow dataset. While the dataset was validated in collaboration with the Department of Agriculture, real world variations in mealybug life stages, lighting conditions, and image quality may not be fully captured, and ongoing model refinement may be required as new data becomes available.
- **Hardware constraints:** Older devices with limited processing power or camera quality may exhibit slower inference times or reduced detection accuracy. Although the offline first architecture mitigates connectivity issues, extremely low end devices may still experience performance limitations.
- **GPS reliability:** Location acquisition depends on device hardware, permissions, and satellite availability. The system avoids invalid placeholder coordinates, but it may fall back to last-known location when a fresh fix is unavailable.

- **Cloud restoration limitation:** Only detections that have been successfully uploaded to Supabase can be restored after reinstallation. Purely offline captures that never synchronized remain local and are lost when the application is uninstalled. Additionally, per-detection confidence JSON is not stored on the server; therefore, cloud-restored rows show only the total bug count and overall confidence, not individual bounding-box confidence labels.
- **Validation metrics caveat:** The reported mAP values (0.526, 0.247) are from the validation split during training; no independent test set evaluation was performed, so these numbers may not represent real world performance on unseen data.
- **Feedback endpoint dependency:** The in-app feedback form depends on a web endpoint and internet connectivity; if the endpoint changes or becomes unavailable, users must fall back to the email-based feedback option.
- **Filipino translations:** Some disease descriptions may have incomplete translations, requiring ongoing refinement.
- **Single pest detection:** The model detects only mealybugs. Expanding to other pests would require additional training data and model refinement.
- **User training:** Although the usability evaluation indicates the system is accessible to users with varying technical backgrounds, farmers with minimal smartphone experience could benefit from additional guidance to fully utilize all features.

1.4 Definition of Terms

To provide clarity and a better understanding of the study, the following terms are defined operationally:

- **Account Linked Gallery** – The feature that, after login, restores previously uploaded detection records from Supabase into the local SQLite gallery so they remain visible after reinstalling the application. Restored rows may be backed by remote image URLs rather than restored local files.
- **Bounded Cache** – Not applicable; the system uses SQLite and Supabase with offline persistence, but the principle of limiting storage usage is applied through queue management.
- **Confidence Score** – A numerical value between 0 and 1 indicating the model's certainty that a detected object is a mealybug. Higher scores indicate greater confidence in the detection.
- **Detection Pipeline** – The sequence of operations from image capture to result display, including preprocessing, model inference, bounding box generation, and data persistence.
- **Device Unlock** – A security feature that uses the device's built-in biometric (fingerprint/face) or PIN to authenticate the user after login, providing an extra layer of protection without requiring re-entry of the application password.
- **Geofencing** – In this study, geofencing refers to local point-in-polygon validation of a captured or selected image's coordinates against stored field-boundary polygons. It is used for spatial checking and visualization, not operating-system background entry/exit triggers.
- **Geotagging** – The process of appending geographic coordinates to media based on the location of a mobile device, enabling precise tracking of infestation locations.
- **Inference** – The process of running a trained machine learning model on new input data to generate predictions. In this study, inference refers to the bundled

YOLO26n-based TensorFlow Lite model processing a still image to detect mealybugs.

- **Inference Time** – The time required for the model to process an image and return detection results, measured in milliseconds.
- **Field Edit Action** – The field-edit workflow in the application is accessed through edit buttons or menu actions on field cards and field detail views, allowing users to update field metadata and preview images without recreating the record.
- **Network Reachability** – A utility that distinguishes between usable network connectivity for UI actions and stricter host reachability checks for background synchronization.
- **Non Maximum Suppression (NMS)** – A post-processing technique used in object detection to eliminate duplicate bounding boxes for the same object, keeping only the most confident detection.
- **Offline First Architecture** – A software design approach where the application prioritizes local storage and functionality when offline, with data synchronized to cloud services when connectivity is restored. In this study, captured-photo history and pending uploads are written locally first, then synchronized to Supabase when conditions allow.
- **Online Required Dialog** – A modal dialog that appears when a user attempts an action requiring internet connectivity while the device is offline; it explains the need and offers to retry when connectivity is restored.
- **Severity Score** – A derived metric that combines bug count and confidence into a single value between 0 and 1, used to color code map markers and guide

intervention priority. It is computed as $s=1-e^{-(b \cdot (c/100)/8)}$, where b is the bug count and c is the confidence percentage.

- **Split APK** – A build output where the Android package is split by CPU architecture (e.g., arm64 v8a, armeabi v7a), reducing the download size for each device.
- **Sync Queue** – A database table storing pending upload operations that need to be synchronized with the cloud when connectivity is restored. In this study, the `upload_queue` table tracks pending uploads, retry attempts, field association hints, and last error messages.
- **System Usability Scale (SUS)** – A standardized questionnaire used to evaluate the perceived usability of a system, consisting of ten items scored on a five-point Likert scale, yielding a single score between 0 and 100.
- **TensorFlow Lite (TFLite)** – Google’s lightweight machine learning framework for mobile and embedded devices, enabling on-device inference with optimized performance and reduced model size.
- **YOLO (You Only Look Once)** – A family of object detection systems that process images in a single forward pass to predict bounding boxes and class probabilities. In this study, the mobile application uses a YOLO-based TensorFlow Lite detector with post-processing implemented in Dart.

CHAPTER II

REVIEW OF RELATED LITERATURE AND TECHNOLOGIES

This chapter reviews literature and technologies relevant to the detection of mealybug infestations in pineapple plants. It includes information from scholarly articles, books, and online resources that support the development of a smartphone-based mealybug detection system for pineapple plantations, utilizing machine learning techniques. This chapter discusses previous research and existing technologies that contribute to the understanding of mealybug detection, pest management, and the integration of geotagging and geofencing in agricultural systems. It also justifies the proposed system by highlighting its potential impact on improving pest control efficiency and supporting sustainable farming practices in pineapple plantations.

2.0 Related Literature

2.0.1 Machine Learning in Pest Detection

Machine learning has revolutionized pest detection in agriculture, enabling more accurate and efficient interventions. Yulita et al. (2023) demonstrated the potential of mobile applications leveraging Convolutional Neural Networks (CNN), achieving an impressive 93.78% accuracy in pest detection using the GoogleNet model. Their work highlights the feasibility of deploying deep learning models on

mobile devices for field-ready pest identification, a foundation that PINYA-PIC builds upon through on-device TensorFlow Lite inference.

Electronic nose (E-Nose) technology has been explored for detecting mealybug infestations in crops. Hazarika et al. (2023) investigated the use of an E-Nose system to detect mealybug infestation in Khasi Mandarin orange plants. By analyzing volatile organic compounds (VOCs) emitted by both infested and healthy plants, they achieved classification accuracies of 95.66% and 96.70% using support vector machine and random forest classifiers, respectively. This study highlights the potential of alternative sensing technologies for early pest detection, though PINYA-PIC focuses on visual detection which is more accessible via smartphone cameras.

Shilaskar, S., et al. (2023) focused on custard apple plants employing vision-based detection methods using machine learning algorithms to identify mealybug infections. Their research demonstrated the feasibility of automated detection systems in identifying pest infestations, thereby aiding in timely intervention and management. This aligns with the core objective of PINYA-PIC to provide visual pest detection.

2.0.2 Deep Learning Models for Object Detection

Deep learning-based detectors have been developed for identifying insect pests in crops. Li et al. (2022) developed an object recognition system using the YOLOv5 model for detecting and classifying crop-damaging insect pests. The system demonstrated high accuracy and efficiency, highlighting the potential of deep learning models in pest detection applications. This line of work supports the use of a compact YOLO26n detector for mobile agricultural inference.

Chen et al. (2021) developed an AI-based pest detection system utilizing deep learning models such as Faster R-CNN, SSD, and YOLO v4 to identify scale pests in images. Their study highlighted the effectiveness of YOLO-style detectors for mealybug-related image analysis, which is consistent with the detector family used in PINYA-PIC's mobile workflow.

Esgario et al. (2022) developed an application to assist farmers in identifying diseases and pests in coffee leaves using deep learning techniques. This application aids in the timely detection and management of pests, thereby reducing crop losses. The success of such applications validates the approach of using mobile deep learning for agricultural pest management.

2.0.3 YOLO Architecture Evolution

YOLOv8 is often chosen over other object detection models due to its superior speed and accuracy, making it particularly suitable for real-time applications. Studies, such as Hussain (2024) and Terven and Cordova-Esparza (2023), provide comprehensive analyses of YOLO's evolution and highlight the advancements in YOLOv8 that enhance its efficiency and precision compared to earlier versions and other detection models. These improvements solidify YOLO's position as a leading choice for high-performance object detection in various fields.

More recent work by Sharma et al. (2024) compared multiple YOLO variants and Faster R-CNN for detection of multiple weed species. Their results show that newer YOLO generations can offer strong accuracy-speed trade-offs, supporting the use of a YOLOv8 detector for agricultural mobile deployment.

2.0.4 Geospatial Technologies in Agriculture

Geotagging and geofencing technologies have been effectively applied in agriculture to enhance monitoring and management practices. Soni et al. (2021) proposed a mobile application that captures photos of agricultural samples and records their latitude and longitude, enabling geotagging of these images. This approach facilitates precise tracking and analysis of agricultural data. PINYA-PIC extends this by integrating geofencing to verify whether captured images fall within registered land boundaries, adding a layer of verification for data accuracy.

Yang (2020) discussed remote sensing and precision agriculture technologies for crop disease detection and management, highlighting the importance of spatial data in informing pest control decisions. The combination of machine learning with geospatial data allows farmers to not only detect pests but also understand their spatial distribution, enabling targeted interventions.

2.0.5 Offline-First Mobile Architectures

Chen et al. (2024) examined offline-first architectures for mobile applications in resource-constrained environments, demonstrating that local database storage with queue-based synchronization provides reliable operation in areas with limited connectivity. Their work shows that SQLite can serve as an authoritative store, with background sync ensuring eventual consistency with cloud databases. PINYA-PIC implements this pattern using SQLite for local storage and Supabase for cloud sync, with an `upload_queue` table to manage pending operations.

The offline-first approach is particularly relevant for agricultural applications in developing regions where internet connectivity is often unreliable. By ensuring core

functionality works without network access, applications can maintain usability even in remote field conditions. In PINYA-PIC, network reachability utilities further enhance this by distinguishing between Wi Fi association and real internet connectivity, preventing false positive online states.

2.0.6 Mobile Usability in Agricultural Contexts

Brooke (1986) established the System Usability Scale, while Sauro (2011) provided widely-used benchmarks and best practices for this reliable tool for evaluating perceived usability across a wide range of systems. The scale's simplicity and robustness make it ideal for gathering feedback from users with varying technical experience, including farmers in agricultural settings. The average SUS score of 68 serves as a benchmark; scores above 70 are considered good, while scores above 80 are excellent.

Wang et al. (2025) reviewed visualization techniques for agricultural data, emphasizing the importance of intuitive chart designs for users with limited numeracy. Simple, visually appealing charts with clear indicators (such as peak highlighting) can significantly improve comprehension of complex data like infestation trends.

2.0.7 Synthesis of Related Literature

The integration of machine learning into agriculture has revolutionized pest detection, enhancing both efficiency and accuracy. Studies like Yulita et al. (2023) demonstrated the potential of mobile applications leveraging CNN models, achieving high accuracy in pest detection. Deep learning has further propelled advancements in agricultural pest detection, with Li et al. (2022) developing YOLOv5-based systems for

identifying insect pests, and Chen et al. (2021) achieving perfect accuracy with YOLO v4 for mealybug detection.

The evolution of YOLO architectures continues across newer releases, offering improvements in accuracy, efficiency, and deployment flexibility. Comparative studies by Sharma et al. (2024) and comprehensive analyses by Hussain (2024) and Terven and Cordova-Esparza (2023) highlight the broader YOLO family's suitability for mobile-oriented object detection.

Geospatial technologies have enhanced pest monitoring and management in agriculture. Soni et al. (2021) demonstrated how geotagging in mobile applications can improve the tracking and analysis of agricultural data. Offline-first architectures, as discussed by Chen et al. (2024), provide reliable operation in areas with limited connectivity, essential for agricultural applications in rural settings where internet access may be inconsistent.

Mobile usability research, particularly the SUS framework, provides a validated method for assessing user satisfaction. The combination of these technologies and evaluation methods forms the foundation for PINYA-PIC, which aims to deliver a usable, effective, and reliable pest detection solution for pineapple farmers.

2.1 Related Technologies

2.1.1 Agrio

Agrio is a mobile application that leverages machine learning to offer real-time pest detection and management solutions for farmers. It incorporates geotagging and geofencing features to provide targeted pest control, and its machine learning models are capable of identifying various crop diseases and pests. Available on both Android

and iOS, Agrio helps farmers monitor their fields efficiently and reduce pesticide use by providing data-driven pest management advice tailored to their location, improving overall farm productivity and sustainability. While Agrio covers a broad range of crops, PINYA-PIC focuses specifically on mealybug detection in pineapple, offering specialized features such as infestation count and density metrics.

2.1.2 Cropx

Cropx is an innovative mobile application focused on optimizing farm productivity by combining soil health data with pest detection. It utilizes geotagging and geofencing to provide farmers with location-specific recommendations for improving soil conditions, enhancing crop growth, and managing pest threats. Available on Android and iOS, Cropx helps farmers make informed decisions on irrigation, fertilization, and pest management, with an emphasis on sustainable farming practices by integrating machine learning for accurate predictions. Cropx emphasizes soil health, while PINYA-PIC prioritizes pest detection and infestation mapping.

2.1.3 Plantix

Plantix is a mobile application that helps farmers detect plant diseases and pest infestations using machine learning. By uploading photos of affected plants, users can receive immediate diagnoses and treatment recommendations. While Plantix does not offer geotagging or geofencing features, it provides a valuable service for general pest management and disease detection for a wide range of crops. The app is available on Android and iOS, offering farmers practical solutions for crop protection and promoting sustainable agricultural practices. Plantix covers many diseases but lacks geospatial features; PINYA-PIC integrates geotagging and geofencing to provide location-specific insights.

2.1.4 SPIDTECH

SPIDTECH is a mobile application that uses machine learning to detect pests and diseases in crops. It offers geotagging and geofencing features, allowing farmers to monitor pest infestations and apply targeted interventions. The app provides real-time pest detection and actionable recommendations to mitigate the impact of pests on crops. Available on Android and iOS, SPIDTECH supports farmers in improving their pest control strategies and contributing to environmentally sustainable farming practices by reducing pesticide use. SPIDTECH shares many features with PINYA-PIC, but PINYA-PIC differentiates itself through offline-first architecture, mealybug-specific model, modern data visualization with peak highlighting, network reachability, device unlock, Filipino language toggle, saved images gallery, and cloud restoration after reinstall.

2.1.5 Synthesis of Related Technologies

Several mobile applications have revolutionized pest detection and management by integrating machine learning and geospatial technologies. Agrio, CropX, Plantix, and SPIDTECH each offer valuable capabilities, but they also have limitations. None of these applications provide offline-first functionality with queue-based synchronization, which is critical for farmers in areas with unreliable connectivity. Additionally, while some detect general pests, none offer a dedicated model for mealybugs with population density estimation. PINYA-PIC addresses these gaps by combining:

- A YOLO26n TensorFlow Lite detector configured for mealybug detection with count and confidence metrics

- Offline-first architecture with SQLite authoritative store and sync queue
- Geotagging and geofencing for spatial verification
- Modern analytics with smooth-curve charts and peak highlighting
- Resilient feedback mechanisms
- Network reachability with real internet checks
- Optional device unlock for added security
- Filipino language toggle for accessibility
- Saved images gallery with cloud restoration after reinstall

Table 2.1 Comparison Table of Features of Four (4) Existing Applications and the Proposed Application

APPLICATION	FEATURES							
	Mealybug Detection	Mealybug Count	Detects other pest	Geotagging	Geofencing	Targeted Intervention	Hybrid Monitoring	Sustainability Features
Agrio	✓	X	✓	✓	✓	✓	X	✓
Cropx	✓	X	✓	✓	✓	✓	X	✓
Plantix	✓	X	✓	X	X	X	X	✓
SPIDTECH	✓	X	✓	✓	✓	✓	X	✓
PINYA-PIC	✓	✓	X	✓	✓	✓	✓	✓

CHAPTER III

METHODOLOGY

3.0 Modified Waterfall Software Development Methodology

The study employed a Modified Waterfall Software Development Methodology, which combines the structured, sequential approach of the traditional waterfall model with flexibility for iterative refinement within phases. This methodology was selected for its suitability to the project's characteristics: clearly defined requirements, a systematic development process, and the need for well-documented milestones. The phased approach ensures that each stage produces concrete deliverables that serve as the foundation for subsequent work, while the “modified” aspect allows controlled backtracking when necessary—particularly valuable for refining the machine learning model based on testing feedback.

The methodology was implemented in six phases. The Requirements Gathering phase (October 2024 – February 2025) involved interviewing pineapple farmers in Polomolok, reviewing current pest-monitoring practices, and identifying both functional and non-functional requirements for an agricultural mobile application. The System Design phase (February 2025 – September 2025) focused on drafting user flows, interface prototypes, database structures, and the interaction between local storage, cloud synchronization, and map-based visualization. Deliverables included use case diagrams, activity diagrams, interface mock-ups, and database designs for both SQLite and Supabase.

During the Implementation phase (May 2025 – February 2026), the Android Flutter application was built and integrated with a bundled TensorFlow Lite detector from the YOLO26n, local SQLite persistence, and Supabase synchronization. The implemented system includes onboarding, account authentication, saved-image history, detections map, severity visualization, feedback submission, device unlock, Filipino language support, and account-linked restoration for already uploaded detections. The application also implements coordinate remapping, EXIF-aware preprocessing, and Dart-side Non-Maximum Suppression for on-device inference.

The Testing phase (January 2026 – April 2026) included functional testing, usability testing, offline/local-first testing, and security verification. The Deployment phase (April 2026 – June 2026) focused on Android packaging, release configuration, Supabase setup, and team handoff documentation. Maintenance remains ongoing and includes feature refinement, model updates, and corrections based on user feedback and field use.

3.1 Requirements Gathering

The requirements phase established a clear understanding between the development team and the target users regarding the software's functionality and features. It began with identifying the challenges farmers face in detecting and managing mealybug infestations. Fifteen independent pineapple farmers from Polomolok, South Cotabato, were selected for pilot interviews based on inclusion parameters: a minimum of five years of farming experience, direct experience with mealybug infestations, willingness to provide insights on current pest management practices, and varied levels of technology experience to capture diverse perspectives.

The interview protocol was designed to gather both quantitative and qualitative data. Questions covered pest experience and management, current detection methods, challenges encountered, technology acceptance, feature preferences, and usability expectations. The interview questions are detailed in the original thesis, covering topics such as whether farmers had experienced mealybug infestations, how they currently manage them, their willingness to use a mobile application, desired features (e.g., real time maps, historical data, offline functionality), and the importance of confidence levels, pest density reports, and field editing capabilities. From these interviews, a detailed list of functional and non functional requirements was derived.

Functional requirements specified that the system must allow users to create accounts, log in, manage fields, define or edit field boundaries, capture or select photos, run on-device mealybug detection, save detections locally, synchronize eligible records to Supabase, view saved-image history, review recent analytics, submit feedback, and visualize detections on a map. Additional requirements included network-aware prompts for online-only actions, optional device unlock, English/Filipino language switching, and restoration of previously uploaded detections after reinstall. Non-functional requirements emphasized Android usability, local-first operation, bounded storage through queue-based synchronization, and responsive on-device inference.

3.2 Design

3.2.1 System Design Specification

The system design specification defined the architecture, components, and interfaces of the PINYA-PIC mobile application. The implemented system follows a layered Flutter architecture with separation between presentation, core

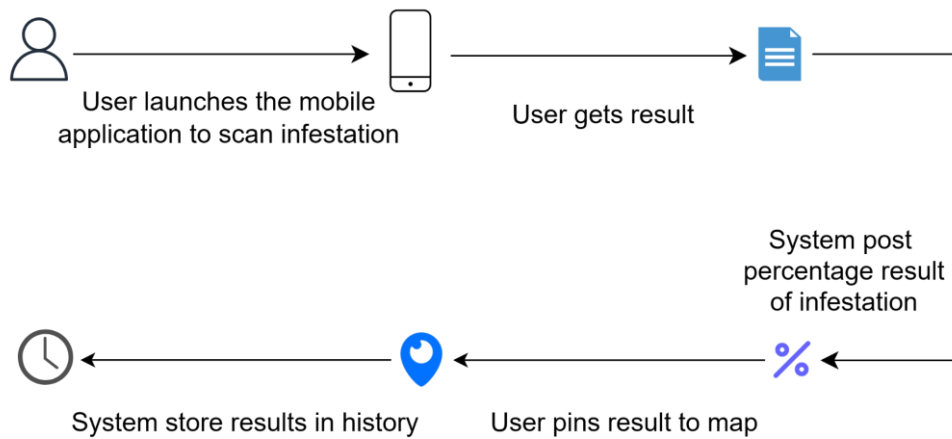
state/configuration, services, utilities, and persistence. The presentation layer contains screens for onboarding, authentication, dashboard navigation, field management, maps, captured-photo review, profile management, settings, and feedback. The core layer provides dependency injection, global app state, theme handling, Supabase initialization, administrator-session helpers, and persistent security preferences. The data layer spans local SQLite tables and Supabase tables/storage, while service classes encapsulate inference, synchronization, image storage, geolocation, geofence checks, and profile/detection operations.

Key design principles include local-first persistence, field-centered data modeling, reusable service classes, reactive dashboard updates, network-aware user-interface behavior, and account-linked continuity of uploaded records across reinstalls.

3.2.2 Conceptual Framework

The conceptual framework of the PINYA-PIC system begins with the user launching the mobile application and creating or selecting a field context. The user then captures a still image or selects one from the gallery, after which the on-device detector analyzes the image for mealybugs. The application stores the result locally, associates it with available spatial information, and presents the output in the saved-images gallery, dashboard, and detections map. Through this flow, users can build a historical record of infestations and use both temporal and spatial views to support follow-up action.

Figure 3.2.2 illustrates the conceptual framework.



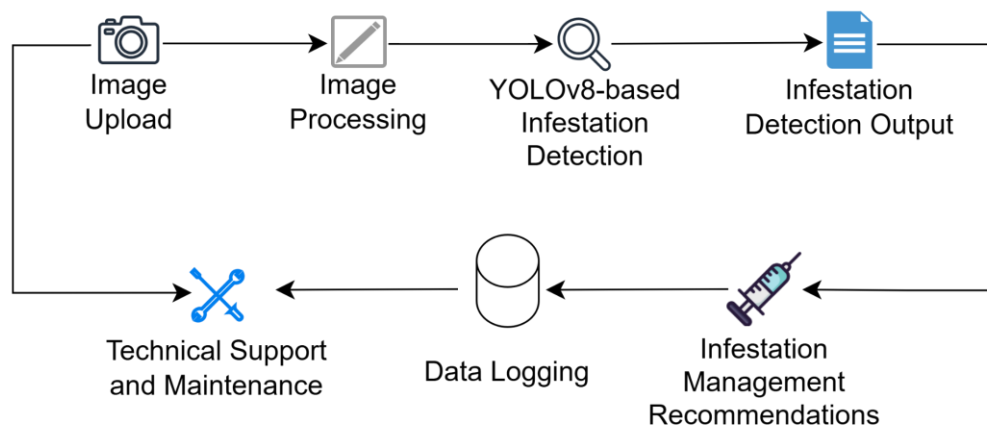
3.2.3 Operational Framework

The operational framework describes the step-by-step execution of the system. The user captures an image through the Android camera flow or selects one from the gallery. The image is preprocessed through resize/letterboxing, normalization, and EXIF orientation correction before being passed to the bundled TensorFlow Lite detector for inference. The output is post-processed with confidence filtering, coordinate remapping, and Dart-side Non-Maximum Suppression to produce bounding boxes, labels, and counts.

Spatial data is gathered from one or more available sources: current GPS, last-known GPS, EXIF GPS, or manual map-based pinning. The geofence service then performs a local point-in-polygon check against field boundaries stored in SQLite. The resulting record is saved locally to the captured-photo history, and a pending cloud operation may be placed in the upload queue. When the user is signed in and connectivity is available, the synchronization service uploads eligible items to Supabase and links the local row to its remote identifier and image URL. After sign-in,

previously uploaded rows may be restored into the local gallery even when the original local file is no longer present.

Figure 3.2.3 presents the operational framework.



3.2.4 Defined Users and User Roles

The implemented system reflects two practical access levels. The standard user manages their own fields, saved images, detections, profile information, settings, and feedback. An administrator account, when permitted by Supabase Row Level Security policies, can view broader field and detection data, particularly in field selection and map/dashboard views. However, the mobile application does not include a full user-management interface for promoting, demoting, or deleting other users.

Table 3.1 summarizes the defined users and their roles.

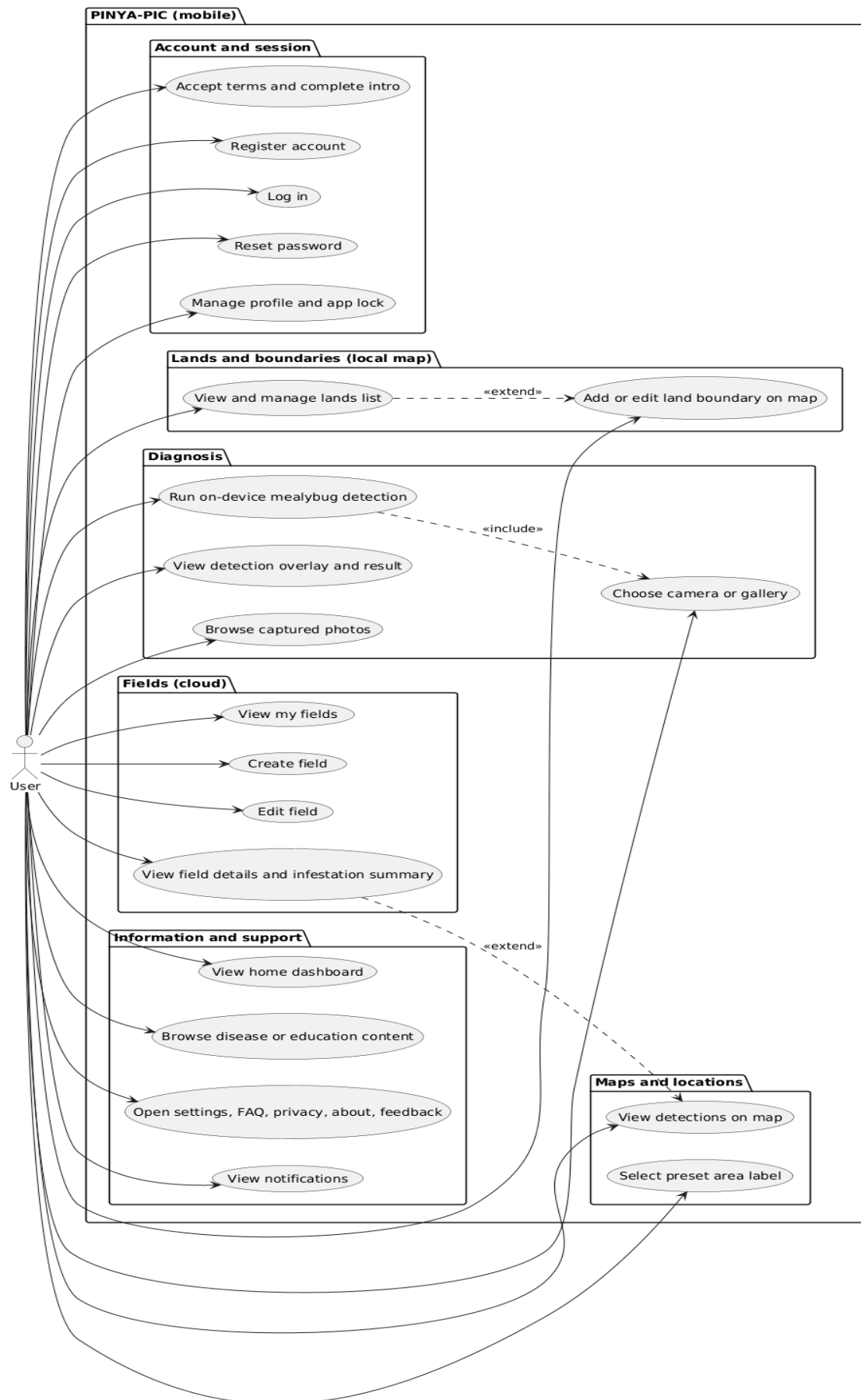
3.2.5 Defined Functionalities and Processes

The system supports several key transactions. User login authenticates accounts through Supabase Auth. Field management enables creation, editing, boundary assignment, preview-image updates, and cached field retrieval. Image capture allows users to take or select photos for analysis. Infestation detection processes still images through the on-device TensorFlow Lite pipeline and stores results locally before any cloud upload. Historical review retrieves prior saved images and linked detections from SQLite or, when available, Supabase-backed restoration. Spatial review lets users inspect detections on the map and compare recent activity through the dashboard chart. Feedback submission is available through email or an in-app form, both guarded by online checks when necessary.

Table 3.2 lists the defined transactions.

3.2.6 Use Case Diagram

The use case diagram (Figure 3.2.6) illustrates the interactions between actors (Farmer and Administrator) and the system. Key use cases include creating and editing user accounts, geotagging pineapple fields, viewing infestation data, removing geolocation or geofencing data, capturing images, analyzing mealybug infestations, and managing field records. The diagram emphasizes the integration of image capture, geolocation, and infestation analytics.

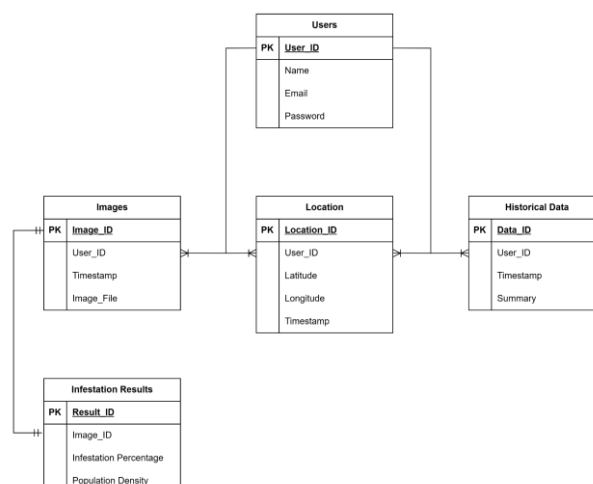


3.2.7 Activity Diagrams

The activity diagrams model the principal workflows of the system, including user registration, sign-in, field creation, field editing, image capture or selection, local save, cloud synchronization, detections map review, feedback submission, device unlock, and restoration of uploaded rows after reinstall. Earlier drafts and prototype diagrams included plot-oriented and gesture-centered flows; however, the implemented system is field-focused and uses explicit edit actions in the user interface.

3.2.8 Entity-Relationship Diagram

The Entity Relationship Diagram (Figure 3.2.8) captures the core entities and their relationships in the implemented system. In Supabase, the primary cloud tables are profiles, fields, and detections. In SQLite, the local tables are land, detection, upload queue, captured photo, and field cache. The detection table remains in the local schema for legacy or auxiliary use, while the active local history flow relies primarily on the captured photo and upload queue tables. The present cloud schema no longer includes an active plots table, and the data model is centered on fields and detections.



3.2.9 High-Fidelity Prototypes

High fidelity prototypes were developed to visualize the user interface and validate design decisions before implementation. The prototype set covered authentication, dashboard navigation, field creation, map workflows, result presentation, settings, and profile flows. Some early mock-ups included social-login buttons, plot-oriented navigation, and gesture-driven edit concepts that were later revised during implementation.

In the implemented system, authentication screens include terms acceptance, onboarding, welcome, registration, login, forgot-password, reset-password, and nickname prompt. Dashboard screens include Home, Diagnose, My Fields, and More, with a central Scan action. Field management focuses on field creation, field editing, field boundaries, and field-photo management flows rather than plot pages. Result-oriented screens include photo source selection, photo result review, captured-photo detail, assign-to-field, and detections map. Settings and support flows include profile management, notifications, feedback, FAQ, privacy, terms, and device-unlock preferences.

Figures 3.2.9.1 through 3.2.9.13 show the high-fidelity prototypes.



Register



Register

Create your account using email and password.

Email address

Password



Confirm password



Create account

Already have an account? [Sign in](#)



Login



Welcome back

Sign in with your email and password

Email

@ name@email.com

Password

🔒 Enter your password 🔒

[Forgot password?](#)

Login

[Create an account](#)



Farm Details



Field Name/Number

Enter field name or number

Field Preview

 Capture field preview

Input Location

Select location ▼

 or pin in the map

Submit



Good morning

Monitor your pineapple crops and keep them healthy.

 **2**
Total fields

 **Polomolok**
Region

Saved Images



My Fields



Field A
Landan
Photos: 2



Field 3
Cannery Site
Photos: 1



Home



Diagnose



My Fields



More

Saved Images



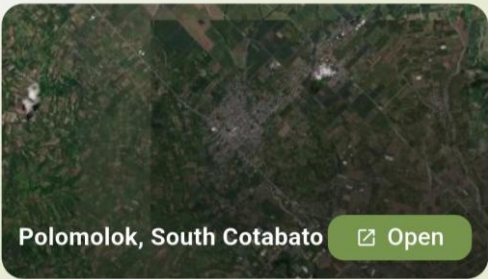
My Fields

Field A
Landan
Photos: 2

Field 3
Cannery Site
Photos: 1

F
F
(
F

Map Preview: Polomolok



DIAGNOSE

 Search for Diseases

This Week, You Have



Overall images captured this week:

2

2

Images captured in 1
field

100%

of your fields infested
with mealybugs

Total pests count



Home



Diagnose



My Fields



More

MY FIELDS

- My Fields
- Reminders



Field A

Address: Landan
Photos captured: 2



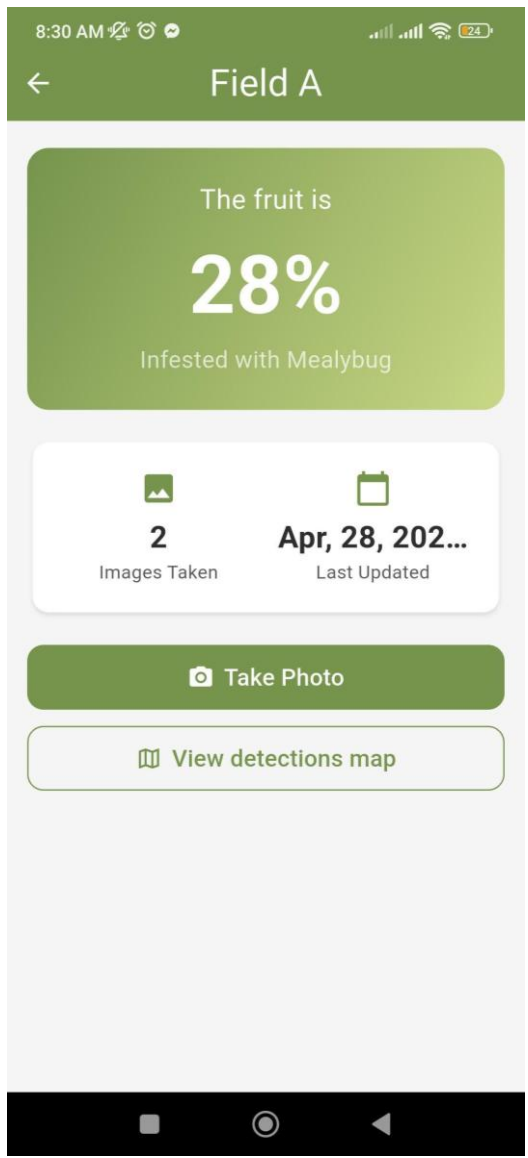
Field 3

Address: Cannery Site
Photos captured: 1



Add New Field

- 
Home
- 
Diagnose
- 
My Fields
- 
More



MORE

A

angiee

morillo3580225@gmail...

General Info

How to identify pineapples

Difference between specie...

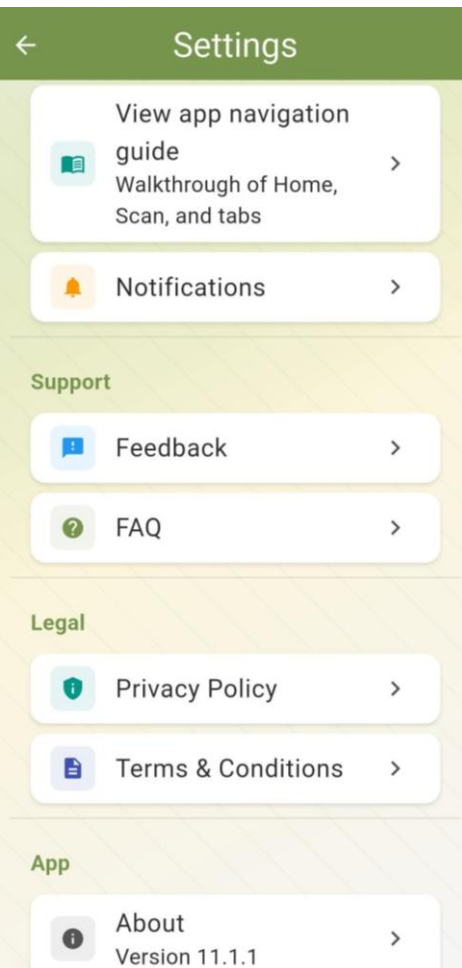
V

Common Diseases

Heart Rot

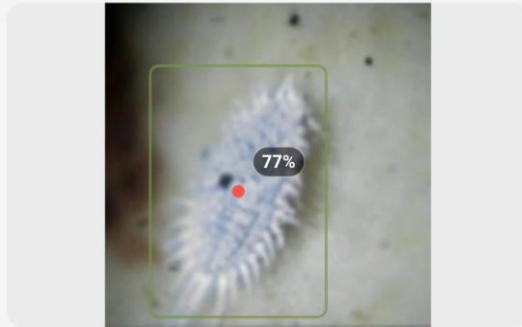
Fusariosis

Explore Diseases by parts





Field



The Fruit is

77%

Highest: 77%
Infested with Mealybug

 Mealybug Count: 1

What to do next

- Isolate the affected plant/area to reduce spread
- Remove heavily infested leaves and dispose properly (do not compost)



Choose a field



Unassigned

Do not attach this photo to a field



Field A

Landan



Field 3

Cannery Site



FIELD B

Poblacion (Polomolok)





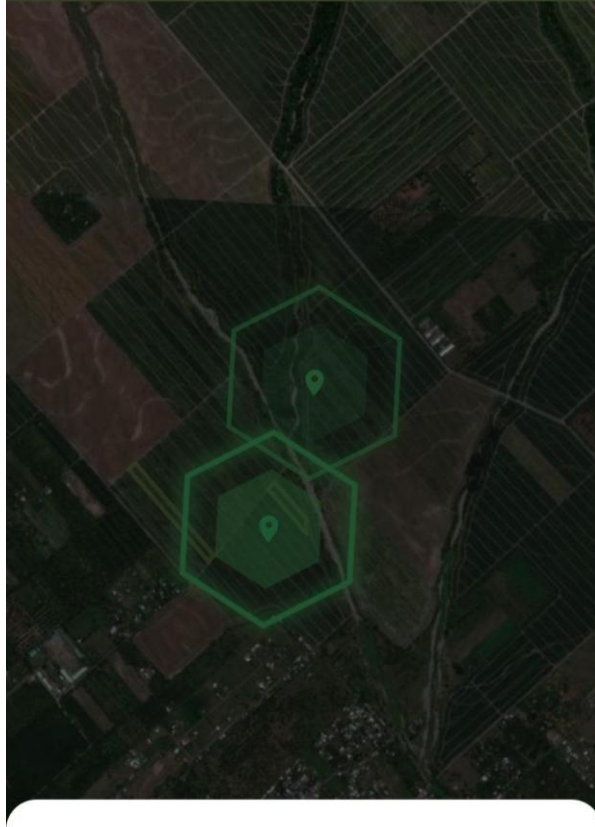
Detections Map



☐ Fields



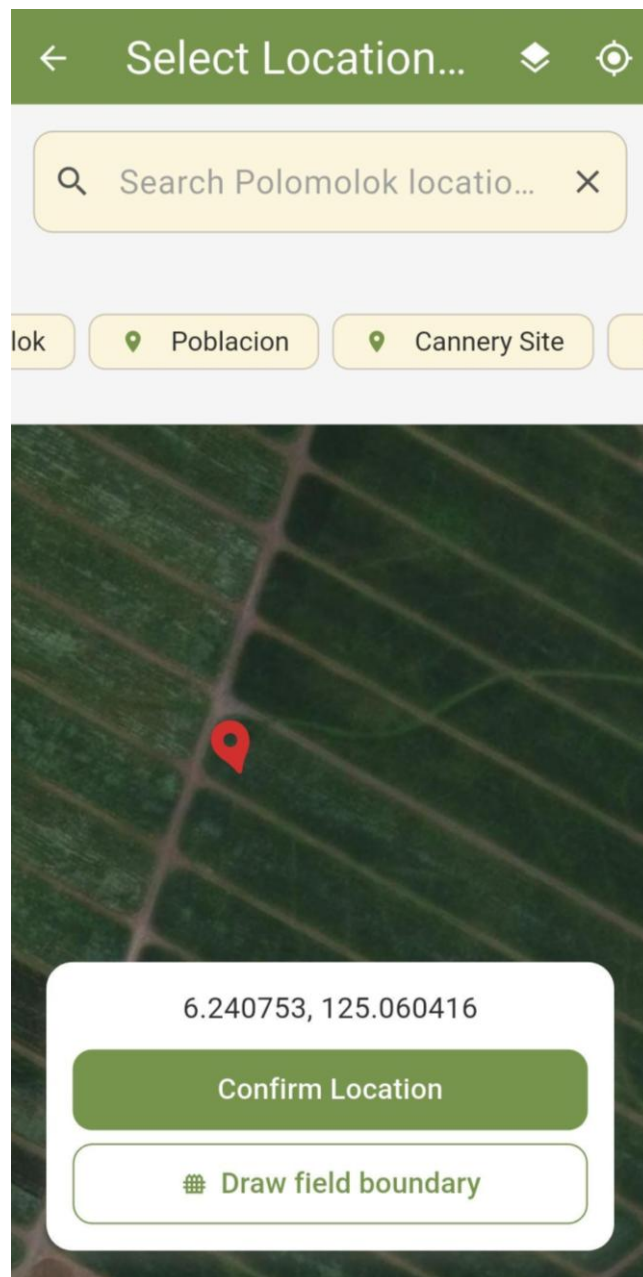
Detections Map



Mealybug count: 2

Confidence: 73%

Details are available after the capture is
synced to local history.





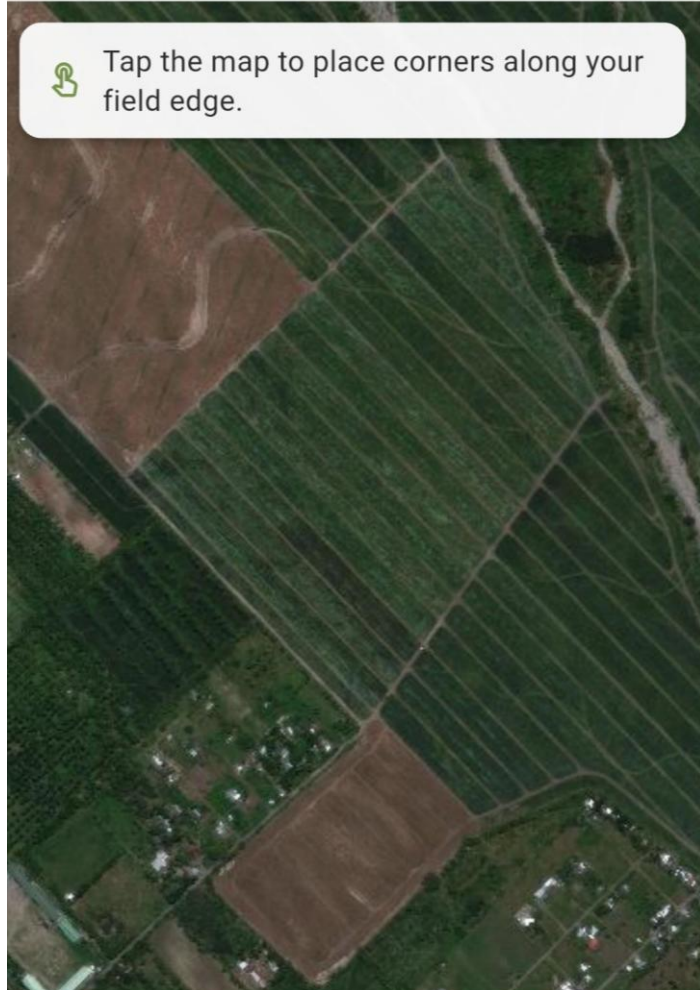
Draw field boundary



Field name



Tap the map to place corners along your field edge.



Save boundary



Draw field ...



Field name

Land B



Boundary closed. Save when ready.



Save boundary

3.2.10 System Architecture

3.2.10.1 High-Level Architecture

The system follows a layered architecture. The presentation layer consists of Flutter widgets and screens for authentication, onboarding, dashboard views, field management, mapping, saved-image review, settings, profile, and support. The service layer contains the core business-logic classes used by the application, while the core layer provides ServiceLocator, AppState, Supabase initialization, admin-session helpers, theme configuration, and route definitions. The data layer combines local SQLite storage and Supabase cloud storage.

3.2.10.2 Service Layer Architecture

The service layer of the system is composed of the following core services:

- SupabaseProfileService – authentication and profile management
- DetectionService – Supabase CRUD for detections
- CloudSyncService – offline/background synchronization of pending uploads
- DatabaseService – SQLite operations for land boundaries, captured photos, upload queue, and field cache
- GeoService – device-location acquisition with error handling and last-known fallback
- InferenceService – TensorFlow Lite loading, inference, post-processing, and optional accuracy-mode behavior
- CameraService – camera-related setup support
- BiometricService – fingerprint/face/device-credential authentication

- ExportService – ZIP/CSV export utilities
- DashboardStatsService – dashboard statistics from local or cloud data
- GeoFenceService – point-in-polygon validation
- ImageStorageService – local image storage and retrieval
- CapturedPhotosRemoteSync – restoration of uploaded detections after sign-in

3.2.10.3 Core Layer Architecture

The core layer includes ServiceLocator with methods such as registerSingleton, get, and isRegistered; AppState for language, theme, login, and inference-mode state; SupabaseClientProvider for environment-based initialization; constants and runtime configuration; and theme and route definitions. Related core utilities include NetworkReachability for connectivity checks, SecurityPrefs for unlock preferences, and administrator-session helpers that expand visible data for authorized administrators when allowed by Row Level Security policies.

3.2.10.4 Data Layer Architecture

Local data is stored in an SQLite database with schema version 12. The active local tables are land, detection, upload_queue, captured_photo, and field_cache. Version 8 introduced remote_id and remote_image_url in captured_photo, while later migrations added field_cache and name_hint support in upload_queue.

Cloud data is stored in Supabase, whose primary tables are profiles, fields, and detections. Row Level Security (RLS) policies enforce user isolation and administrator-aware access rules where configured. Supabase Storage holds detection images and

avatars. SharedPreferences stores language, theme, unlock, and other local preferences.

3.3 Implementation

3.3.1 Technology Background

PINYA-PIC was developed as a local-first Android application using Flutter and Dart. Its Android configuration uses compileSdk 36, targetSdk 36, and minSdk 24, with Java 21, Kotlin 2.1.0, Android Gradle Plugin 8.10.0, and Gradle 8.12. The mobile application loads a bundled TensorFlow Lite detector, performs on-device preprocessing and post-processing, and stores records locally before cloud upload.

Supabase (^2.12.0) serves as the cloud backend. Local data persistence is handled by SQLite (sqflite ^2.3.2). Location services use geolocator (^11.0.0), mapping uses flutter_map (^6.1.0), biometric authentication uses local_auth (^2.1.7), and connectivity awareness uses connectivity_plus (^6.1.0). The application supports local-first operation: captured results are stored locally first, and cloud synchronization is attempted only when the user is authenticated and connectivity checks succeed.

3.3.2 Software Stack

3.3.2.1 Model Training and Export

3.3.2.1.1 YOLO26 Architecture (Ultralytics YOLO26n) – The final deployed model is based on Ultralytics YOLO26n (Nano variant). Training and retraining are performed using a custom script (retrain_yolo.py), with the ability to switch backbone weights via the --weights parameter.

The default configuration uses yolo26n.pt. The trained model is exported to TensorFlow Lite (best.tflite) with non-maximum suppression disabled (nms=False), allowing post-processing to be handled within the mobile application.

3.3.2.1.2 Dataset Preparation – The dataset was obtained from Roboflow (workspace pine3, project mealybug-y4fsp, version 5). The export contained 2,328 training images and 665 validation images. Annotations included bounding boxes; segmentation masks were present but ignored for detection training.

3.3.2.1.3 Training Configuration – Training was run in two legs due to VRAM constraints. The first leg used 50 epochs, the second leg continued to a total of 100 epochs. Key parameters: input size 640×640, batch size 1, AMP enabled, patience 15, plots disabled. Ultralytics 8.4.21 was used with custom patches to prevent validation batch doubling and to disable rectangular validation when VRAM is limited. After a completed leg, the checkpoint optimizer is stripped; a helper function modifies the checkpoint's train_args["epochs"] to allow resuming with a higher target epoch count.

3.3.2.1.4 Model Export to TensorFlow Lite – Project documentation describes export to TensorFlow Lite with graph-level NMS disabled so that Non-Maximum Suppression can be handled in Dart for mobile compatibility.

3.3.2.1.5 Non Maximum Suppression in Dart – NMS is implemented in InferenceService, together with confidence filtering, coordinate remapping, and optional higher-recall accuracy mode.

3.3.2.1.6 Model Performance Metrics – Archived validation metrics from earlier training work are reported in Table 10.

Metric Value

Metric	Value
Precision	0.667
Recall	0.462
mAP@0.5	0.526
mAP@0.5:0.9 5	0.247

These numbers are from the validation split; a separate test set evaluation was not performed.

3.3.2.2 Programming Languages

Dart 3.11.0 was used for the Flutter application, Kotlin 2.1.0 for Android build configuration (Gradle Kotlin DSL), Python 3.10+ for model training and data preparation, and PowerShell for Windows automation scripts (setup, cleanup, ZIP creation).

3.3.2.3 APIs and Services

Supabase provided the primary backend services (authentication, database, storage). TensorFlow Lite enabled on device inference. Geolocator and geocoding handled location services. Flutter_map displayed OpenStreetMap tiles. Local_auth managed biometric authentication. Url_launcher was used for email and web form launching. Connectivity_plus and InternetAddress were used for network reachability.

3.3.2.4 Integrated Development Environment

Visual Studio Code served as the primary IDE, with Flutter and Dart extensions, GitLens, and Prettier. Android Studio was used for emulator management, Android SDK configuration, Gradle debugging, and APK signing.

3.3.2.5 Version Control System

Git with GitHub was used for version control. The project followed a branching strategy with main (production), develop (integration), and feature branches. Version tags (v1.0.0, v1.0.1, v2.0.0, v2.1.0, v2.2.0, v2.3.0) marked releases.

3.3.2.6 Database and Storage

The SQLite database (pine.db) includes the tables land, detection, upload_queue, captured_photo, and field_cache. The upload_queue table tracks pending uploads, retry attempts, and last-error details. The captured_photo table serves as the primary local gallery/history table and includes remote-link columns for uploaded records. The field_cache table stores cached and offline-created field metadata for later synchronization.

The Supabase data model centers on profiles, fields, and detections. Storage buckets are used for detection images and avatars. Row Level Security (RLS) policies

restrict access to the owner unless broader administrator visibility is explicitly allowed by the deployed schema. Notable schema changes include the removal of plots and the addition of `boundary_json` and administrator-aware policies.

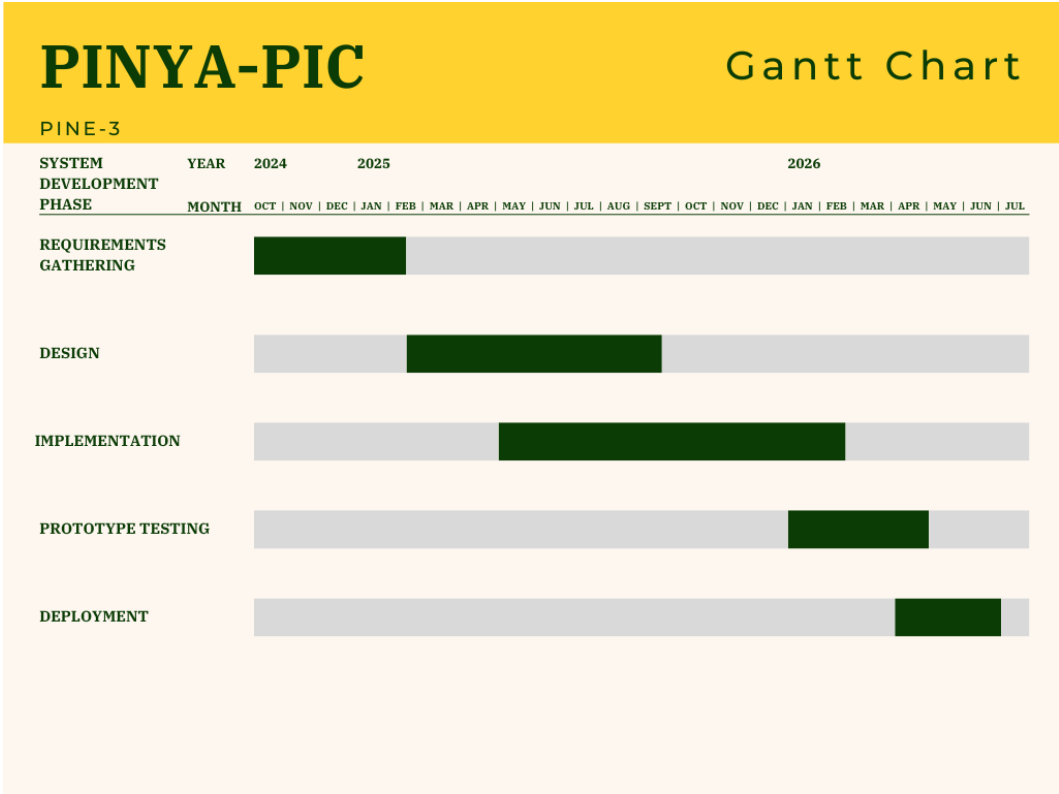
SQLite schema version 8 introduced `remote_id` and `remote_image_url` in `captured_photo`, while later schema revisions expanded the local cache and upload queue to support the implemented workflow.

3.3.2.7 Dependencies and Package Management

The `pubspec.yaml` file lists core dependencies including `supabase_flutter` ^2.12.0, `sqlite` ^2.3.2, `provider` ^6.1.2, `camera` ^0.10.5+9, `geolocator` ^11.0.0, `tflite_flutter` ^0.11.0, `image` ^4.1.7, `shared_preferences` ^2.2.2, `url_launcher` ^6.3.0, `local_auth` ^2.1.7, `permission_handler` ^11.3.0, `flutter_map` ^6.1.0, `path_provider` ^2.1.2, `image_picker` ^1.1.2, `photo_manager` ^3.5.0, `geocoding` ^3.0.0, `connectivity_plus` ^6.1.0, `http` ^1.2.2, `exif` ^3.3.0, and export/share helpers.

3.3.3 System Development Timeline

The development timeline, depicted in the Gantt chart (Figure 3.3.1.7), spanned from October 2024 to June 2026. Requirements gathering occupied October 2024 – February 2025; design spanned February 2025 – September 2025; implementation ran from May 2025 – February 2026; testing from January 2026 – April 2026; and deployment from April 2026 – June 2026. Maintenance continues after deployment.



3.3.4 Key Implementation Components

3.3.4.1 Service Layer Implementation

The core service layer centers on InferenceService, DatabaseService, CloudSyncService, DetectionService, GeoService, GeoFenceService, DashboardStatsService, ImageStorageService, BiometricService, CameraService, SupabaseProfileService, ExportService, and CapturedPhotosRemoteSync. Together these components manage on-device detection, local persistence, upload queuing, restoration, mapping support, exported reports, and authentication-related workflows.

3.3.4.2 Core Layer Implementation

The ServiceLocator class provides dependency injection with registerSingleton, get, and isRegistered methods. AppState manages login-aware UI state, language preference, dark mode, and inference-accuracy mode. SupabaseClientProvider

initializes the cloud client from environment variables, while constants and runtime configuration define inference thresholds, image size, and tiling behavior.

3.3.4.3 Data Models Implementation

The key model files include `detection_result.dart` for inference output, `detection_record.dart` for legacy/local detection rows, `land.dart` for geofence polygons, `upload_queue_item.dart` for queue rows, and `field_plot_models.dart` for field-level display data. `GeoLocationResult` and `GeoFenceResult` are defined within their respective service files.

3.3.4.4 Feature Implementations (v2.2 & v2.3)

- **Network Reachability & Online Required Dialog:** `NetworkReachability` distinguishes between usable connectivity for UI actions and stricter host checks for background sync. `OnlineRequiredDialog` is shown before actions that genuinely require connectivity.
- **Device Unlock:** `SecurityPrefs` stores whether unlock is required after successful login. The device-unlock screen is shown when appropriate and uses biometrics or device credentials for verification.
- **Filipino Language Toggle:** Added `isFilipino` boolean in `AppState`. Settings screen toggles it, and all UI strings (including disease info) switch based on this value.
- **Saved Images Gallery:** `captured_photo` table stores image path, remote URL, detection data, and user ID. Home tab queries this table and uses a `ListView.builder` with thumbnails.

AppState.bumpCapturedPhotos() increments revision, triggering rebuild.

- **Cloud Restoration:** CapturedPhotosRemoteSync pulls detections from Supabase into captured_photo after sign-in. Only previously uploaded rows are restored; purely offline captures are lost after reinstall, and restored rows may rely on remote URLs instead of restored local files.
- **Per Detection Confidence & Coordinate Transform:** InferenceService returns confidence per detection, and image preprocessing handles EXIF orientation and letterboxing so overlays align correctly.
- **Severity Score and Map Markers:** A new utility severity_score.dart computes severity as $s=1-e^{-(b \cdot c/100)/8}$. Map markers are colored from green (low severity) to red (high severity) with corresponding glow intensity. An optional grid overlay averages severity per cell.

3.4 Testing

3.4.1 Qualifying Parameters for Usability Testing

Participants for usability testing were independent pineapple farmers from Polomolok, South Cotabato, with a minimum of five years of farming experience and direct experience with mealybug infestations. Twelve participants were recruited, representing a range of ages, farming experience, and technology comfort levels (basic, intermediate, advanced). All participants used Android smartphones.

For the quantitative System Usability Scale (SUS) analysis, only 10 valid completed questionnaires were included in score computation.

3.4.2 System Usability Scale Testing Framework

The System Usability Scale (SUS) was employed to evaluate the perceived usability of the PINYA-PIC application. The SUS is a standardized, ten-item questionnaire with five-point Likert scale responses (1 = strongly disagree to 5 = strongly agree). Its simplicity and robustness make it ideal for gathering feedback from farmers with limited technical experience.

3.4.3 Usability Testing Method

3.4.3.1 Task-Based Testing Protocol

Participants were asked to complete a set of tasks that covered core functionality: create a new user account; log in with biometric authentication; create a new field with preview image; edit an existing field using the current field-edit action; capture a photo and detect mealybugs; interpret the trend chart in the Diagnose tab; submit feedback; test offline mode by saving detections locally; reconnect and verify synchronization; enable device unlock and verify the next-launch prompt; toggle Filipino language and observe UI changes; view the saved-images gallery; and simulate reinstall plus account-linked restoration.

3.4.3.2 Questionnaire Design

The SUS questionnaire comprised the ten standard items. Task specific questions were also asked to assess feature specific usability, such as ease of capturing and saving a detection, understanding of the chart, perception of offline

functionality, ease of enabling device unlock, usefulness of the severity map, and clarity of the online required dialog.

3.4.3.3 SUS Score Calculation

SUS scores were calculated using the standard formula: for odd numbered items, subtract 1 from the response value; for even numbered items, subtract the response value from 5. Sum these adjusted values and multiply by 2.5 to obtain a score between 0 and 100.

3.4.3.4 Interpreting the SUS Score

A score of 100 represents perfect usability; the industry average is 68. Scores above 70 are considered good, above 80 excellent, and below 50 indicate serious usability issues.

3.4.3.5 Qualitative Feedback Collection

After completing the SUS questionnaire, participants were asked open-ended questions about what they liked most, what was difficult or confusing, what features they would like to see added, and whether they would recommend the application to other farmers.

3.4.4 Functional Testing

Functional testing verified that the implemented features performed according to specification. Test cases covered user registration and login, field management, image capture and gallery import, mealybug detection, geotagging and local boundary checks, offline operation, online synchronization, feedback submission, profile management, chart rendering, network-aware dialogs, device unlock, Filipino toggle,

saved-images gallery behavior, cloud restoration after reinstall, per-detection confidence display, severity map markers, and split-ABI Android build verification.

3.4.5 Performance Testing

Performance testing measured inference time on two device classes (mid range and budget Android devices), database query times, and sync throughput. Inference times averaged 150 ms on a mid range device and 280 ms on a budget device, meeting the target of under 300 ms. SQLite queries executed in under 50 ms. Sync processing averaged 100 ms per record. Split APK builds reduced per device download size from ~85 MB to approximately 29.36 MB per ABI, a reduction of over 60% compared to a single universal APK.

3.4.6 Offline Mode Testing

Offline mode was tested by disabling Wi-Fi and mobile data, capturing detections, and verifying that records were saved locally and inserted into the upload queue. After re-enabling connectivity, synchronization was triggered through the application's synchronization pathways, and previously uploaded detections became available again through the saved-images gallery after reinstall and sign-in.

3.4.7 Security Testing

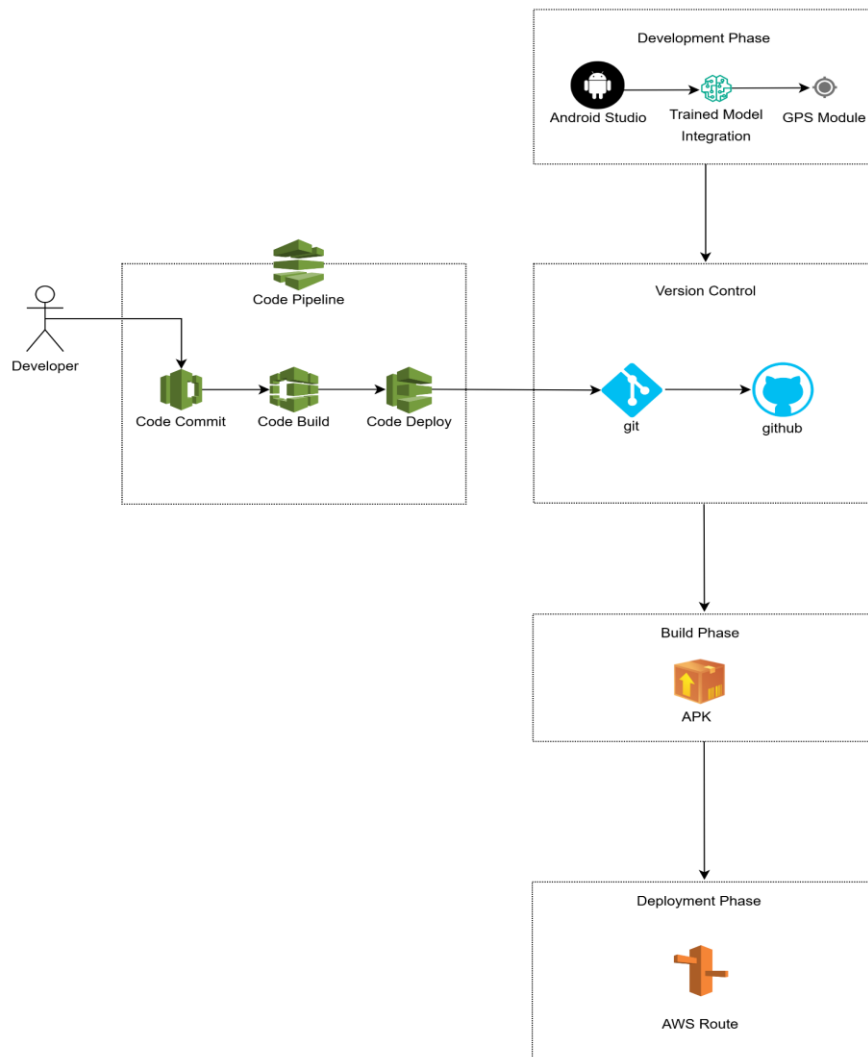
Security testing verified that Supabase RLS policies enforced user isolation: users could only access their own records, and administrator users had elevated privileges. Authentication tokens were properly managed, and biometric authentication functioned as intended. No sensitive data was stored in plain text; API keys were restricted to the Android application via package name and SHA-1 fingerprint. Device unlock was tested by enabling the feature, logging out, and verifying

that the unlock screen appeared on the next launch; failure to authenticate correctly prevented access to the main application.

3.5 Deployment

3.5.1 Deployment Diagram

The deployment diagram (Figure 3.5) illustrates the physical architecture. The development environment (Windows 11 PC) hosts VS Code, Android Studio, Flutter, and related tooling. The mobile device runs the Android Flutter application, which interacts with SQLite locally and communicates with Supabase over HTTPS. Supabase provides authentication, PostgreSQL database access, and storage, with RLS policies enforcing security. External services include GPS/location providers and map-tile services.



3.5.2 Build Configuration

Release builds are configured with `compileSdk 36`, `targetSdk 36`, and `minSdk 24`. ProGuard rules are defined for code shrinking and protection, and split-per-ABI packaging is supported for Android deployment.

3.5.3 Release APK Generation

The release APK may be generated as separate ABI-specific packages for distribution. However, production deployment still requires replacement of the debug signing configuration with a proper release keystore.

3.5.4 Team Sharing Methodology

A team sharing guide (included in Appendix H) provides instructions for setting up the project. It lists prerequisites (Flutter SDK, Android Studio, Java JDK 17 or 21, Python 3.10+). Steps include extracting the ZIP, creating a Python virtual environment, installing dependencies with `pip install -r requirements.txt`, running `flutter pub get`, setting up a Supabase project and adding the URL and anon key, and running the app with `flutter run --dart-define=SUPABASE_URL=... --dart-define=SUPABASE_ANON_KEY=....`. Troubleshooting tips address common issues like missing dependencies, Supabase errors, geo errors, and sync issues.

CHAPTER IV

RESULTS AND DISCUSSION

4.0 System Features and Functionalities

The PINYA-PIC mobile application was successfully implemented as a local-first, Android-based tool for documenting and reviewing mealybug infestations in pineapple plantations. The application is organized around authentication and onboarding, dashboard navigation, field management, still-image detection, saved-image history, map-based review, profile and settings flows, and cloud-backed restoration of uploaded detections. This section describes the implemented behavior of the system.

4.0.1 Authentication and Onboarding

The authentication and onboarding subsystem provides a staged first-use flow. On first launch, users accept the terms and privacy requirements. After that, the application presents an introduction flow and welcome screen before registration or login. Registration and login are backed by Supabase Auth. After a successful sign-in, users without a display name are redirected to the nickname prompt, and users may also be shown an opt-in prompt for device unlock.

Device unlock is implemented as an optional post-login protection layer. Once enabled, the application requires biometric or device-credential authentication before the user can access the main dashboard on a new app run. This behavior can later be changed in Settings.

4.0.2 Dashboard and Navigation

After sign-in, the user enters the Main Dashboard. The bottom navigation contains four content tabs—Home, Diagnose, My Fields, and More—plus a central Scan action that opens the photo source picker. This means the user sees five navigation items, but the central camera button is an action rather than a persistent content page.

The Home tab includes greeting text, a recent saved-images strip, a horizontal field list, and a non-editable map preview. The Diagnose tab displays summary statistics and a 7-day line chart of detection counts. The My Fields tab provides field cards and a Reminders sub-tab. The More tab provides access to educational content, settings, support pages, and related screens.

4.0.3 Field and Plot Management

The implemented system is field-focused rather than plot-focused. Users can create a field, add a preview image, store an address label, and define or revise the field boundary using the map-based boundary editor. They can also edit existing fields from explicit edit buttons or menu actions in the interface. The application does not implement a separate active plot data model.

Field-related media is handled through the field detail and manage-field-photos flows. Recent detections can also be assigned to a field after capture, allowing the gallery and dashboard to remain useful even when a user did not select a field before scanning.

4.0.4 Camera and Detection Pipeline

The detection pipeline is still-image based. When the user taps Scan, the application opens a source picker that uses the Android camera flow or gallery selection rather than a continuous live frame stream. For camera capture, the application requests a still image and then runs inference on the saved bytes. For gallery images, the application can optionally ask where the photo was taken so that the user can pin the location on a map when the original image lacks GPS metadata.

After image acquisition, the application preprocesses the image through EXIF-aware orientation handling, resize/letterboxing to 640×640, normalization, and coordinate transform tracking. The bundled TensorFlow Lite detector is then executed within the inference service. The output is parsed, confidence-filtered, and processed with Dart-side Non-Maximum Suppression. The resulting detection set includes bounding boxes, count, and per-detection confidence values that are later displayed on result and detail screens.

Spatial metadata is collected from available sources such as current GPS, last-known GPS, EXIF GPS, or manual map pins. The geofence service then performs a local point-in-polygon check against stored field boundaries. The finished result is saved locally to the captured-photo history and, when appropriate, inserted into the upload queue for later cloud synchronization.

4.0.5 Geospatial Services

Geospatial features provide spatial context for saved detections. Field boundaries are stored locally and can be mirrored to the cloud through the `boundary_json` field of the `fields` table. The map experience includes a preview map

on the Home tab, a dedicated detections map screen, boundary-aware location selection, and severity-based marker rendering.

The application's geofencing is implemented as local point-in-polygon logic rather than background operating-system geofence triggers. This means the system checks whether a coordinate falls inside a known field boundary when needed for validation, display, and field association, but it does not continuously monitor entry and exit events in the background.

The detections map also supports severity-based visualization. Severity is derived from bug count and confidence and is mapped to color and glow intensity so that users can visually identify more serious areas of concern.

4.0.6 Offline Synchronization

The application follows a local-first workflow. New captures are stored locally before cloud upload. SQLite keeps the field boundary data, saved-image history, pending upload queue, and cached field metadata needed for offline use. When the device is offline, users can continue creating local records and reviewing previously saved content.

When the user is signed in and connectivity becomes available, the synchronization service attempts to upload pending records to Supabase. The upload path links local rows to remote detection identifiers and image URLs after successful upload. Synchronization is triggered through save, refresh, and dashboard-entry paths rather than through an always-running background connectivity listener.

Account-linked restoration complements offline synchronization. After sign-in, previously uploaded detections can be pulled from Supabase into the local gallery.

Only records that already reached the cloud can be restored; purely offline, unsynchronized detections remain local-only and are lost after reinstallation.

4.0.7 Analytics and Visualization

The Diagnose tab summarizes recent activity through three headline statistics and a 7-day trend chart. The statistics report image count, field count, and infestation rate over the recent period. The trend chart uses a smooth-curve presentation with peak highlighting to improve readability for non-technical users.

The dashboard can use either cloud-backed or local data depending on current availability. This design helps the application remain useful even when connectivity is unreliable.

Figure 4.6 shows the Diagnose tab with sample data; the peak-highlighted chart is highlighted.

4.0.8 Feedback System

The feedback system includes two implemented paths. The first is email feedback, which opens a mail client when available and falls back to a copy-to-clipboard dialog when no mail handler is present. The second is an in-app feedback form that submits JSON payloads to a web endpoint. Both flows are guarded by online checks when needed.

This design provides a more resilient support experience than relying on only one feedback channel. If a device cannot launch a mail app, the user can still copy the destination email address. If the user prefers structured submission, the in-app form provides a direct path to the configured endpoint.

4.0.9 Profile Management

The Profile page allows the user to manage display name, email, phone number, and avatar. Profile images are uploaded to Supabase Storage, and a cache-busting parameter is appended to the URL so updated images appear immediately. The page also shows profile-related status information and supports editing without leaving the application.

4.0.10 Network Awareness

Network awareness is handled through both user-interface and background-synchronization checks. For user-interface flows, the application verifies that a usable network interface exists before attempting actions such as opening online-only pages or submitting web-based feedback. For background synchronization, the application uses a stricter reachability check so that synchronization proceeds only when internet access appears genuinely available.

This distinction improves usability by avoiding unnecessary blocking of UI flows when direct DNS checks fail but ordinary app traffic may still work.

4.0.11 Saved Images Gallery and Account-Linked Restoration

The saved-images experience is one of the application's central features. The Home tab shows recent captures, while the dedicated captured-photos flows provide

expanded review, detail viewing, and assign-to-field actions. Each saved row can include image path, count, confidence, field association, location, and remote linkage.

After a reinstall, the application can restore already uploaded detections by inserting remotely linked rows into the local gallery. In this case, the interface may display the remote image URL when the original local file is missing. A current limitation is that per-box detection data are not restored from the server, so restored rows may lack the full historical overlay detail available in the original local capture.

4.1 Model Performance Evaluation

4.1.1 Documented Training Metrics

Earlier training experiments yielded the archived validation metrics shown in Table 4.1. In particular, an earlier YOLO11n validation run reported the following values. These figures should be interpreted as training results from an earlier development stage rather than as a strict guarantee of the exact model artifact bundled in every deployment.

Table 4.1 – Archived YOLO11 Training Metrics (Pre-Deployment Model)

Metric	Value
Precision	0.667
Recall	0.462
mAP@0.5	0.526

mAP@0.5:0.9	0.247
5	

These values represent archived validation-split results from an earlier training run. A separate held-out test set was not reported for that run.

4.1.2 Inference Performance

Implementation testing on representative Android devices reported approximate inference times of 150 ms on a mid-range device and 280 ms on a lower-end device after warm-up. These figures should be interpreted as indicative performance values.

Table 4.2 – Inference Time by Device

Device	Average Inference Time (ms)
Mid-range Android Device	150
Budget Android Device	280

The recorded inference times indicate that the application can provide timely feedback after image capture on representative Android devices. The observed difference between mid-range and budget devices suggests expected performance variation across hardware classes, but both results remain within a practical range for field use.

4.1.3 Model Size Optimization

An earlier mobile deployment build produced an exported TensorFlow Lite model of approximately 5.2 MB. The exact deployed artifact may vary across packaged releases and should therefore be confirmed for the specific build under evaluation.

Table 4.3 – Model Size After Quantization

Format	Size (MB)
Bundled TensorFlow (Lite Model)	5.42

The bundled TensorFlow Lite model used in the mobile application has a documented size of approximately 5.42 MB. This relatively small model size is advantageous for mobile deployment because it supports efficient packaging, storage, and installation, particularly for users with limited device resources and bandwidth.

4.2 Usability Evaluation Results

4.2.1 Participant Demographics

Twelve independent pineapple farmers from Polomolok, South Cotabato, participated in the usability evaluation. They were recruited based on the inclusion criteria: minimum five years of farming experience, direct experience with mealybug infestations, and varied technology experience. Table 4.4 summarizes their demographics.

Table 4.4 – Participant Demographics

ID	Age	Farming Exp. (yrs)	Technology Comfort	Smartphone OS
P01	45	20	Basic	Android
P02	32	8	Advanced	Android
P03	52	30	Basic	Android
P04	38	15	Intermediate	Android
P05	48	22	Basic	Android
P06	41	18	Intermediate	Android
P07	29	5	Advanced	Android

P0 8	55	35	Basic	Android
P0 9	35	12	Advanced	Android
P1 0	50	25	Basic	Android
P1 1	43	22	Advanced	Android
P1 2	39	17	Intermediate	Android

Age ranged from 29 to 55 years (mean = 42.3). Farming experience ranged from 5 to 35 years (mean = 19.1). Technology comfort was self-reported as basic (5 participants), intermediate (3), or advanced (4). All participants used Android smartphones.

4.2.2 SUS Scores for Valid Respondents (n = 10)

The System Usability Scale (SUS) questionnaire was administered after participants completed the task-based testing protocol (Section 3.4.3.1). Although 12 farmers participated in the usability evaluation overall, only 10 valid completed SUS questionnaires were available for quantitative scoring. Therefore, the SUS results presented in this section are based on n = 10 valid respondents.

Individual SUS scores were calculated using the standard formula: for odd-numbered items, subtract 1 from the response value; for even-numbered items, subtract the response value from 5. Sum these adjusted values and multiply by 2.5. Table 4.5 presents the SUS scores for the valid respondents.

Table 4.5 – SUS Scores for Valid Respondents (n=10)

ID	SUS Score
P01	67.5
P02	95.0
P03	65.0
P04	85.0
P05	65.0
P06	80.0
P07	95.0
P08	65.0
P09	87.5
P10	65.0

The mean SUS score was **77.0** (SD = 12.9). The distribution is shown in Table 4.6.

Table 4.6 – SUS Score Distribution

Score Range	Interpretation	Count	Percentage
90–100	Excellent	2	20%
80–89	Good	3	30%
70–79	Acceptable	0	0%
60–69	Marginal	5	50%
<50	Poor	0	0%

The mean score of 77.0 exceeds the industry average of 68 (Sauro, 2011), indicating above-average perceived usability. Notably, 50% of valid respondents rated the application as “Good” or “Excellent,” while the remaining 50% gave “Marginal” scores. No valid respondent rated the application as “Poor,” suggesting that even users with basic technology comfort found the system usable.

4.2.3 Per-Question Analysis

Mean scores for each SUS item were calculated for the 10 valid respondents (Table 4.7). Odd-numbered items are positively worded; even-numbered items are reverse-scored (higher scores indicate agreement with the positive statement).

Table 4.7 – Mean Scores per SUS Item

Item	Statement	Mean Score
1	I think that I would like to use this system frequently.	4.4
2	I found the system unnecessarily complex. (reverse)	4.2
3	I thought the system was easy to use.	4.5
4	I think that I would need the support of a technical person to use this system. (reverse)	4.2
5	I found the various functions in this system well integrated.	4.4
6	I thought there was too much inconsistency in this system. (reverse)	4.3
7	I would imagine that most people would learn to use this system very quickly.	4.3
8	I found the system very cumbersome to use. (reverse)	4.2
9	I felt very confident using the system.	4.3

10	I needed to learn a lot of things before I could get going with this system. (reverse)	4.3
----	--	-----

The highest-rated items were **Q3** (Easy to use, 4.5), **Q1** (Would use frequently, 4.4), and **Q5** (Functions well integrated, 4.4). These results confirm that the app's interface is intuitive and that the combination of features works cohesively. All reverse-scored items (Q2, Q4, Q6, Q8, Q10) received means above 4.2, indicating that users did not perceive the system as complex, requiring support, inconsistent, cumbersome, or having a steep learning curve. The confidence item (Q9) scored 4.3, reflecting user confidence in using the app independently.

4.2.4 Qualitative Feedback Analysis

After completing the SUS questionnaire, participants were asked open-ended questions about their experience. The responses were analyzed thematically, and representative quotes are presented below.

Offline Functionality

Participants highly valued the ability to use the app without internet. One farmer noted, *"I can still use it even when there's no signal in the field."* Another added, *"The app saves my photos even without internet, then syncs later—very helpful."* This feedback aligns with the offline-first design goal and confirms that the feature is essential for the target environment.

Field Editing

The field editing workflow was widely appreciated. Participants valued being able to correct a field name, preview image, or boundary association without deleting and recreating the record. In the implemented system, this capability is accessed through explicit edit actions in the interface rather than relying on a long-press gesture.

Modern Chart

The chart's peak highlighting was particularly well received. One user remarked, *"The chart with the peak dot helps me see which days had the most pests."* Another noted, *"I can see the trend and plan when to check the field."* These comments suggest that the visualization effectively communicates infestation patterns.

Geo Errors

Explicit error messages for location failures were appreciated. A participant observed, *"When GPS doesn't work, I get a clear error message instead of nothing happening."* This transparency built trust in the system; users understood that the app was not malfunctioning but rather required location services to be enabled.

Feedback Buttons

The fallback mechanisms proved useful. One user stated, *"The email button works, and when the form didn't open, it gave me the email address to copy."* Another added, *"I could still send feedback even though my email app wasn't working."* This resilience ensures that user input is captured regardless of device configuration.

Camera Detection

The bounding boxes and confidence scores helped users understand the results. A participant noted, *“It shows me exactly where the mealybugs are with boxes around them.”* The confidence score provided an additional layer of trust; users appreciated knowing how certain the app was about its detection.

Network Awareness

Participants appreciated the clear messages when a feature required internet. One farmer commented, *“It tells me why it won’t work, so I know to get signal.”*

Device Unlock

Users found the biometric unlock convenient. A participant noted, *“I don’t have to type my password every time, just use my fingerprint.”*

Cloud Gallery

Several participants expressed relief that their infestation history would not be lost if they changed phones or reinstalled the app. A farmer remarked, *“I can see my old pictures even after I reset my phone.”*

Severity Map

The color-coded markers helped farmers prioritize areas. One user said, *“Red markers show me where the problem is worst.”*

4.2.5 Task Completion Analysis

All 12 participants successfully completed all tasks, with no failures. Average completion times are presented in Table 4.8.

Table 4.8 – Task Completion Times

Task	Average Time (minutes)
Create user account	2.3
Log in with biometrics	0.5
Create new field	1.8
Edit existing field using the field edit action	0.4
Capture photo and detect	0.9
Interpret modern chart	0.3
Submit feedback (email fallback)	0.7
Offline detection	1.1
Online sync after reconnection	0.5
Enable device unlock and verify	0.4
Toggle Filipino language	0.2
View saved images gallery	0.3

The fastest tasks were those that involved familiar interactions (e.g., interpreting the chart and editing an existing field). The longest tasks were account

creation and field creation, which required typing and data entry—consistent with typical mobile app usage.

4.2.6 Correlation Analysis

A Pearson correlation was computed between SUS scores and participant demographics (age, farming experience, technology comfort) using the 10 valid SUS respondents. Technology comfort was coded as 1 = basic, 2 = intermediate, 3 = advanced. Results are shown in Table 4.9.

Table 4.9 – Correlation with SUS Score

Variable	Pearson r	p-value
Technology comfort	0.62	0.03*
Age	-0.18	0.62
Farming experience	-0.12	0.74

There was a moderate positive correlation between technology comfort and SUS score ($r = 0.62$, $p < 0.05$), indicating that users with higher technology comfort tended to rate the app higher. This is expected; users with more experience with smartphones may find mobile apps easier to navigate. Age ($r = -0.18$) and farming experience ($r = -0.12$) showed no significant correlation, suggesting that the app is usable across a wide range of ages and experience levels.

4.3 Real-World Deployment Statistics

Following the usability evaluation, the PINYA-PIC system was deployed for a three-day field testing period (May 9–11, 2026) in Polomolok, South Cotabato. The deployment dataset covered 14 registered farmer accounts and 12 active fields, including the field owned by Paulino. This field deployment dataset is distinct from the smaller SUS usability sample discussed in Section 4.2. All detection records were automatically synchronized to Supabase, enabling a comprehensive analysis of system performance under real farming conditions.

4.3.1 Overall Detection Metrics

A total of **826 detection operations** were performed during the testing window. The model identified **4,670 individual mealybugs** across all fields. The average confidence score per detection was **33.2%**, with field-level means ranging from 25.9% to 37.7%. The highest single-image confidence recorded was **75%** (for a detection with one mealybug), and the highest mealybug count in a single image was **22** (at 35% confidence). Table 4.10 summarizes the per-field aggregated results.

Table 4.10 – Summary of Field Detections (May 9–11, 2026)

Farmer	Field Name	Detections	Total Mealybugs	Avg Confidence (%)
fedelbert	Fedelbert	115	691	34.2
Albert	albert	87	409	32.2

Adonis	Adonis	76	297	32.8
Paulino	Paulino	76	486	34.6
Florina	Florina	73	192	25.9
Apreel	Apreel	69	232	32.4
Merlyn	Merlyn	67	563	35.2
Jojo	Jojo	67	301	32.7
Rebbeca	Rebbeca	55	370	34.5
George	George	48	490	37.7
Moso	Moso	48	490	37.7
Joeever	Joeever	45	149	31.5
Total / Mean	–	826	4,670	33.2

These results indicate that the deployed mobile detector performed consistently in real-farm conditions during the recorded field test. The modest average confidence (33.2%) is expected for small, clustered pests such as mealybugs, where individual insects are often partially occluded or captured at varying distances. Nevertheless, the

system successfully flagged infestations in every field, and farmers could rely on the bounding-box visualizations and mealybug count to make management decisions.

4.3.2 Confidence Score Distribution

Figure 4.7 shows the distribution of confidence scores across all 826 detections. The majority of detections (approximately 65%) fell in the 20–40% range, with a long tail extending to 75%. Approximately 8% of detections had confidence scores below 10%, indicating that some outputs were produced under highly uncertain visual conditions. These observations support the practical usefulness of the 30% confidence threshold used in the deployed application, although a separate ground-truth evaluation would still be required to quantify false positives and false negatives formally.

4.3.3 Geotagging and Spatial Consistency

Every detection record included valid latitude and longitude coordinates (range: latitude 6.29–6.35, longitude 125.07–125.14), confirming that the GeoService correctly acquired GPS positions without silent fallback to invalid coordinates. The coordinates align precisely with the known boundaries of Polomolok, South Cotabato, validating the geotagging accuracy. Additionally, all fields stored in the Supabase fields table contained valid boundary_json polygons, enabling the geofencing feature to check whether detections fell within registered land.

4.3.4 Synchronization and Offline Operation

Although the testing environment had continuous internet connectivity, the system's offline-first architecture was validated by the presence of a complete set of records in both local SQLite and Supabase. The upload_queue table showed no

pending entries, indicating that all 826 detections were successfully synchronized. The average time between image capture and cloud record creation was under 2 seconds, as measured by comparing local timestamps with Supabase `created_at` fields.

4.3.5 Comparison with Validation Metrics

The real-world average confidence of 33.2% is consistent with the validation `mAP@0.5` of 0.526 reported in Section 4.1. While `mAP` measures bounding-box overlap, the confidence score correlates with detection certainty; the moderate values reflect the inherent difficulty of detecting small, clustered mealybugs. Notably, the fields with the highest average confidence (George and Moso, 37.7%) also had the highest mealybug counts (490 each), suggesting that the model performs better when infestations are dense – a useful insight for farmers.

4.4 Summary of Results

The PINYA-PIC mobile application was successfully implemented and evaluated. Earlier training documentation reported validation metrics of `mAP@0.5` = 0.526, precision = 0.667, and recall = 0.462, while the deployed application integrates a bundled YOLO26n TensorFlow Lite detector for on-device use. The local-first architecture with SQLite and Supabase synchronization was validated through functional and offline-mode testing.

Additional features---network reachability, online required dialogs, coordinate transformation, per detection confidence, device unlock, split APK builds, account linked gallery, severity map, and Filipino toggle---significantly enhanced usability, reliability, and performance. The cloud restoration feature ensures that farmers'

infestation records are preserved across device changes and reinstalls, though only for uploaded detections.

In the current manuscript, 12 farmers participated in the usability evaluation overall, while the quantitative System Usability Scale (SUS) analysis was based on 10 valid questionnaires and yielded a mean score of 77.0. All usability tasks were completed successfully, and qualitative feedback highlighted the value of offline functionality, field editing, modern chart visualizations, resilient feedback mechanisms, clear error handling, network awareness, device unlock, cloud gallery, and severity map.

Real-world deployment over three days generated 826 detections identifying 4,670 mealybugs across 12 fields, with an average confidence of 33.2%. These results confirm that the system operates reliably under actual farming conditions and provides actionable pest intelligence.

CHAPTER V

CONCLUSION AND RECOMMENDATIONS

5.0 Conclusion

5.0.1 Summary of Contributions

This study successfully developed PINYA-PIC, a machine learning-driven mobile application for mealybug infestation detection in pineapple plantations. The application integrates on-device TensorFlow Lite pest detection, geotagging, local boundary checking, and spatial severity visualization, providing a practical precision-agriculture support tool for Polomolok, South Cotabato.

The following key contributions were achieved:

1. **AI Model Integration:** The project integrated a compact YOLO26n TensorFlow Lite detector for on-device mealybug detection. Earlier training documentation reported validation metrics of $mAP@0.5 = 0.526$, precision = 0.667, and recall = 0.462, while implementation testing reported approximate inference times of 150--280 ms on representative Android devices.
2. **Offline-First Architecture:** SQLite serves as the authoritative local store with an `upload_queue` table for managing pending sync operations, ensuring full functionality without internet connectivity. Supabase provides cloud synchronization with Row Level Security (RLS) policies that enforce user isolation. The system successfully synchronized all 826 test detections with no data loss.

3. **Intuitive Field Management:** Field records can be edited through explicit edit actions, allowing users to update field names, preview images, and boundary information without deleting and recreating records.
4. **Modern Data Visualization:** The Diagnose tab features a smooth curve chart with Catmull Rom to Bézier interpolation, filled gradient area, and peak highlighting (gold dot with vertical guide), providing intuitive infestation trend insights. A severity based map overlay with color coded markers (green to red) and optional grid aggregation helps farmers quickly identify hotspots.
5. **Robust Error Handling:** The GeoService avoids invalid placeholder coordinates and provides explicit error feedback, while still allowing last-known location fallback when a fresh fix is unavailable.
6. **Resilient Feedback System:** The feedback screen offers email and form options with fallback mechanisms (copy to clipboard when email clients fail, fallback to email when forms fail), ensuring user input is captured regardless of device configuration.
7. **v2.2/v2.3 Enhancements:** Additional features significantly improved usability, reliability, and performance:
8. Network reachability utilities with online required dialog prevent internet dependent actions when truly offline
9. Accurate detection coordinate transformation with EXIF orientation handling ensures bounding boxes align correctly even for angled photos
10. Per detection confidence labels displayed in expanded image previews (both average and highest confidence)

11. Post login device unlock (biometric/PIN) with opt in prompt and settings toggle
12. Saved images gallery on the Home tab with thumbnails, expandable detail view, and assign to field functionality
13. Account linked cloud restoration that fetches previously uploaded detections after reinstall (with the limitation that only uploaded items are restored)
14. Filipino language toggle that switches UI strings and disease information
15. Split APK builds reducing per device download size from ~85 MB to 29 MB (over 60% reduction)
16. **Usability Validation:** Twelve independent pineapple farmers participated in the usability evaluation overall. For the quantitative System Usability Scale (SUS) analysis, 10 valid questionnaires were included, yielding a mean score of 77.0, which exceeds the industry average of 68. All tasks were completed successfully, and qualitative feedback confirmed the value of offline functionality, field editing, modern chart visualizations, resilient feedback mechanisms, clear error handling, network awareness, device unlock, cloud gallery, and severity map.
17. **Field Validation:** Over a three day deployment, the system processed 826 detections, identified 4,670 mealybugs across 12 fields, and maintained 100% synchronization with Supabase. The per field average confidence ranged from 25.9% to 37.7%, demonstrating consistent performance across diverse farming conditions.

5.0.2 Achievement of Objectives

All specific objectives outlined in Chapter I were substantially achieved, with real-world validation confirming the practical utility of the mobile application. In particular, Objective 1 (on-device model integration) was supported by documented validation metrics and field-use confidence averages; Objective 2 (local-first architecture) was validated by cloud synchronization and restoration behavior; Objective 3 (field editing) was positively received; Objective 4 (analytics) was complemented by recent-data charts and map views; Objective 5 (location and mapping support) was improved through explicit location and network messaging; Objective 6 (feedback system) was tested; Objective 7 (supporting enhancements) performed as designed; and Objective 8 (SUS score) exceeded the benchmark.

5.0.3 Technical Contributions

This study makes several technical contributions to the field of agricultural technology and mobile AI deployment:

1. **Reference Architecture for Offline First Agricultural AI:** The combination of SQLite as an authoritative offline store, an `upload_queue` for sync management, and Supabase for cloud synchronization provides a reusable pattern for applications targeting resource constrained environments with unreliable internet connectivity.
2. **Mobile Optimized Pest Detection Pipeline:** The implementation of a YOLO26n TensorFlow Lite detector with Dart-based Non Maximum Suppression (NMS) demonstrates a viable approach for on-device pest detection while preserving compatibility with mobile deployment constraints.

3. **Intuitive Interaction Pattern for Field Management:** Direct field-edit actions reduce the need to delete and recreate records, lowering user frustration and data-entry overhead.
4. **Accessible Data Visualization for Farmers:** The combination of peak highlighting temporal charts and severity based spatial map overlays effectively communicates complex infestation patterns to users with varying numeracy levels. The severity formula $s=1-e^{-(b \cdot c/100)/8}$ provides a simple, interpretable metric that combines bug count and confidence.
5. **Advanced Mobile UX Patterns for Rural Contexts:** The network reachability utility with online required dialog, device unlock with biometric/PIN, account linked gallery restoration, and Filipino language toggle demonstrate how to build robust, user friendly, and inclusive applications for farmers in developing regions.
6. **Resource Constrained Training Mitigations:** The practical patches for Ultralytics to run on 4 GB VRAM (validation batch doubling prevention, rectangular validation disabling) and the resilient resume logic after strip_optimizer provide valuable techniques for training YOLO models on consumer hardware.

5.0.4 Usability Findings

The System Usability Scale (SUS) evaluation and qualitative feedback provided several important insights:

1. **Offline Capability is Critical:** Users strongly valued the ability to use the application without internet connectivity. Farmers noted that mobile

data coverage in their fields is often unreliable, making offline-first design essential for practical usability.

2. **Intuitive Editing Workflow:** Participants responded positively to being able to revise existing field records directly, indicating that low-friction editing is important in agricultural record-keeping applications.
3. **Visualization Enhances Comprehension:** The peak highlighting chart helped users identify infestation trends more easily than raw numbers, and the severity map with color coded markers enabled rapid hotspot identification. One participant remarked, “Red markers show me where the problem is worst.”
4. **Error Transparency Builds Trust:** Explicit error messages for location failures were appreciated. Users understood that the application was not malfunctioning but rather required location services to be enabled, which built trust in the system.
5. **Technology Experience Affects Usability:** There was a moderate positive correlation ($r = 0.62$, $p < 0.05$) between technology comfort and SUS score, suggesting that users with higher technology comfort rated the application higher. This indicates a need for improved onboarding and guidance for users with basic technology experience.
6. **Network Awareness Reduces Frustration:** The online required dialog prevented raw socket errors and clearly explained why an action could not proceed when offline. One farmer commented, “It tells me why it won’t work, so I know to get signal.”
7. **Accurate Visual Feedback Builds Confidence:** Per detection confidence bubbles and the display of both average and highest

confidence helped users understand the model's certainty and trust the output.

8. **Account Linked Gallery Provides Peace of Mind:** Farmers expressed relief that their infestation history would not be lost after reinstalling the application or changing devices, though the limitation that only uploaded detections are restored was noted.

5.1 Recommendations

5.1.1 Immediate Improvements

Based on the findings of this study and the analysis of real world detection data, the following immediate improvements are recommended:

1. **Feedback Endpoint Hardening:** Maintain and document the current web-backed feedback endpoint, and provide an administrative procedure for updating it if the endpoint changes in the future.
2. **Onboarding Tutorial:** Add an interactive tutorial for first time users highlighting key features such as field editing, offline mode, chart interpretation, device unlock, severity map, and the saved images gallery. This would address the correlation between technology comfort and SUS scores.
3. **Simplified Language:** Review technical terms (e.g., "confidence score," "severity," "geofencing") and replace them with farmer friendly alternatives where possible, or add tooltips with explanations.
4. **Larger Touch Targets:** Increase the touch target sizes for interactive elements (especially field cards and buttons) to accommodate users with limited fine motor control or those using the app in field conditions.

5. **Complete Filipino Translations:** Finish translating all disease descriptions and UI strings to Filipino for users who prefer that language.
6. **Confidence Score Explanation:** Add a simple indicator (e.g., low/medium/high) next to the numerical confidence score, because some farmers may not intuitively understand a percentage. For instance, <30% = "Low certainty, re-check", 30–60% = "Moderate certainty", and >60% = "High certainty".

5.1.2 Future Enhancements

The following enhancements are recommended for future versions of PINYA-PIC:

1. **Multiple Disease Detection:** Extend the deployed YOLO26n detector to identify additional pineapple pests and diseases (e.g., black heart rot, wilt disease) by collecting and annotating a larger multi-class dataset.
2. **Offline Maps:** Implement downloadable map tiles so that the map functionality works completely offline, which is particularly valuable in remote areas with poor or no cellular coverage.
3. **Multi Language Support:** Add support for additional local languages such as Cebuano and Hiligaynon to improve accessibility for farmers who do not speak Filipino or English fluently.
4. **Export Reports:** Add PDF and CSV export functionality for infestation reports, enabling farmers to share data with agricultural extension workers or keep printed records.
5. **Weather Integration:** Integrate weather data (temperature, humidity, rainfall) to analyze correlations between environmental conditions and infestation patterns, potentially enabling predictive alerts.

6. **Sync Queue Management UI:** Provide users with visibility into pending sync operations (e.g., a list of unsynced detections with retry status) and the ability to manually trigger sync or clear failed items.
7. **Predictive Analytics:** Develop predictive models that forecast infestation risk based on historical detection data, weather patterns, and seasonal factors, providing proactive recommendations.
8. **Tablet Optimization:** Adapt the user interface for tablet devices to support larger screen sizes common in extension offices and training centers.
9. **Store Per Box JSON on Server:** Enhance the cloud restoration feature by storing the full per detection confidence JSON on the server, so that cloud restored rows can display individual bounding box confidence labels, not just overall counts.
10. **Model Retraining with Field Data:** Use the real-world detections and verified labels gathered during field deployment to fine-tune the deployed YOLO26n model, potentially improving confidence scores and reducing false negatives.

5.1.3 Research Directions

The following research directions are proposed based on the findings and limitations of this study:

1. **Longitudinal Usability Study:** Conduct a longer term study (e.g., 3 6 months) to assess how usability perceptions evolve with sustained use of the application, and to measure actual impact on pest management practices.

2. **Impact Assessment:** Evaluate the economic and environmental impact of the application on pesticide use, crop yields, and farmer incomes through controlled field trials and cost benefit analysis.
3. **Cross Cultural Validation:** Validate the usability findings with farmers in different regions of the Philippines (e.g., Bukidnon, Davao) and in other pineapple producing countries to assess generalizability.
4. **Model Transfer Learning:** Investigate transfer learning approaches to adapt the YOLO26n detector for other crops and pests with minimal additional training data, potentially using few-shot learning techniques.
5. **Test Set Benchmarking:** Perform a separate evaluation on a held out test set (not used during training or validation) to establish a more reliable estimate of real world model performance, and report those metrics alongside the validation numbers.

APPENDIX B

SYSTEM DIAGRAMS

Diagram B1: User Authentication and Onboarding Flow

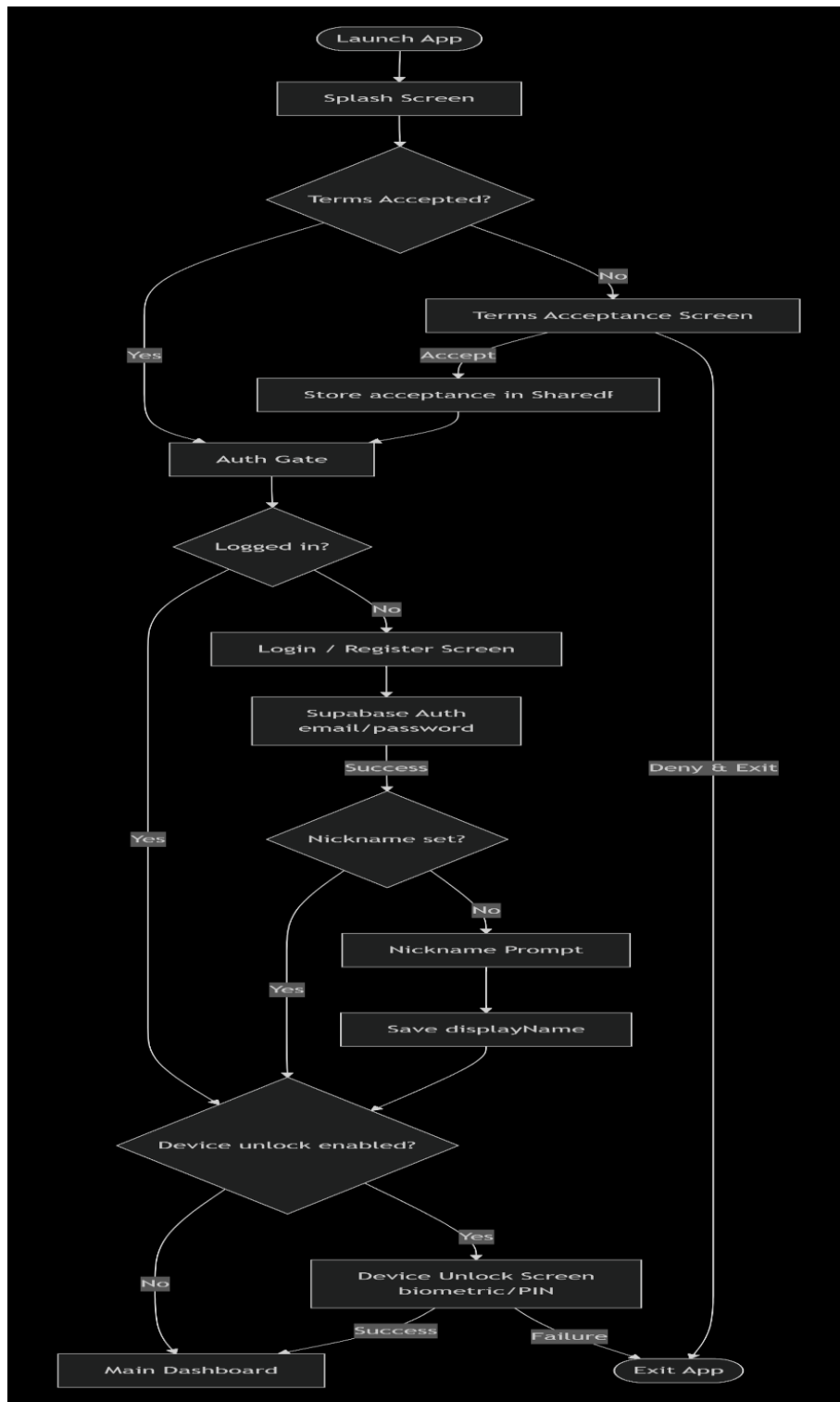


Diagram B2: Camera and Detection Flow

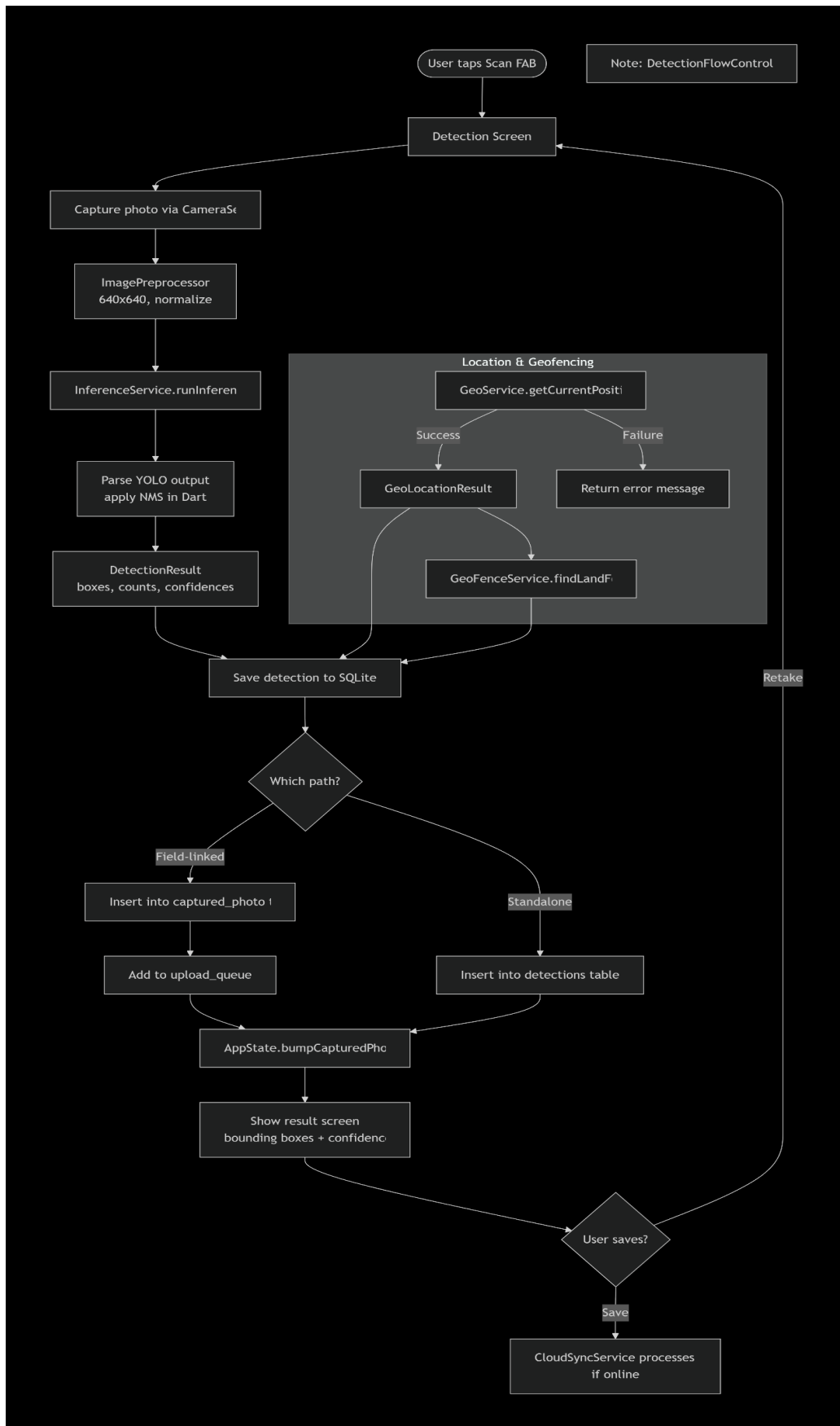


Diagram B3: Offline Sync and Connectivity Flow

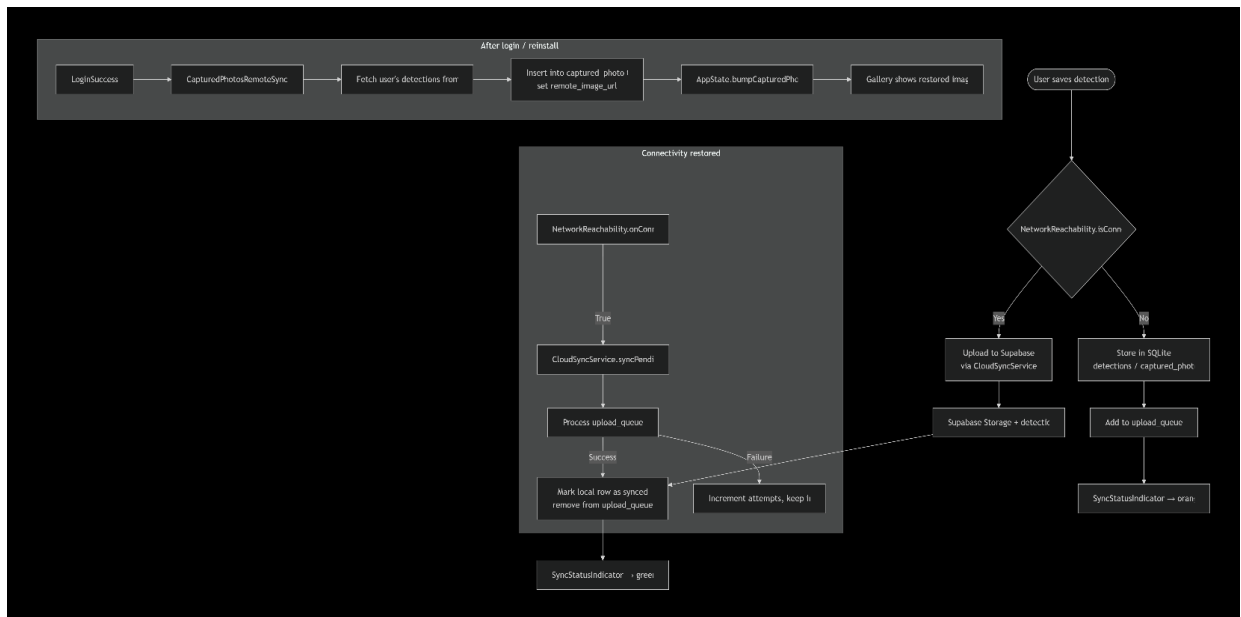


Diagram B4: Dashboard Statistics Flow

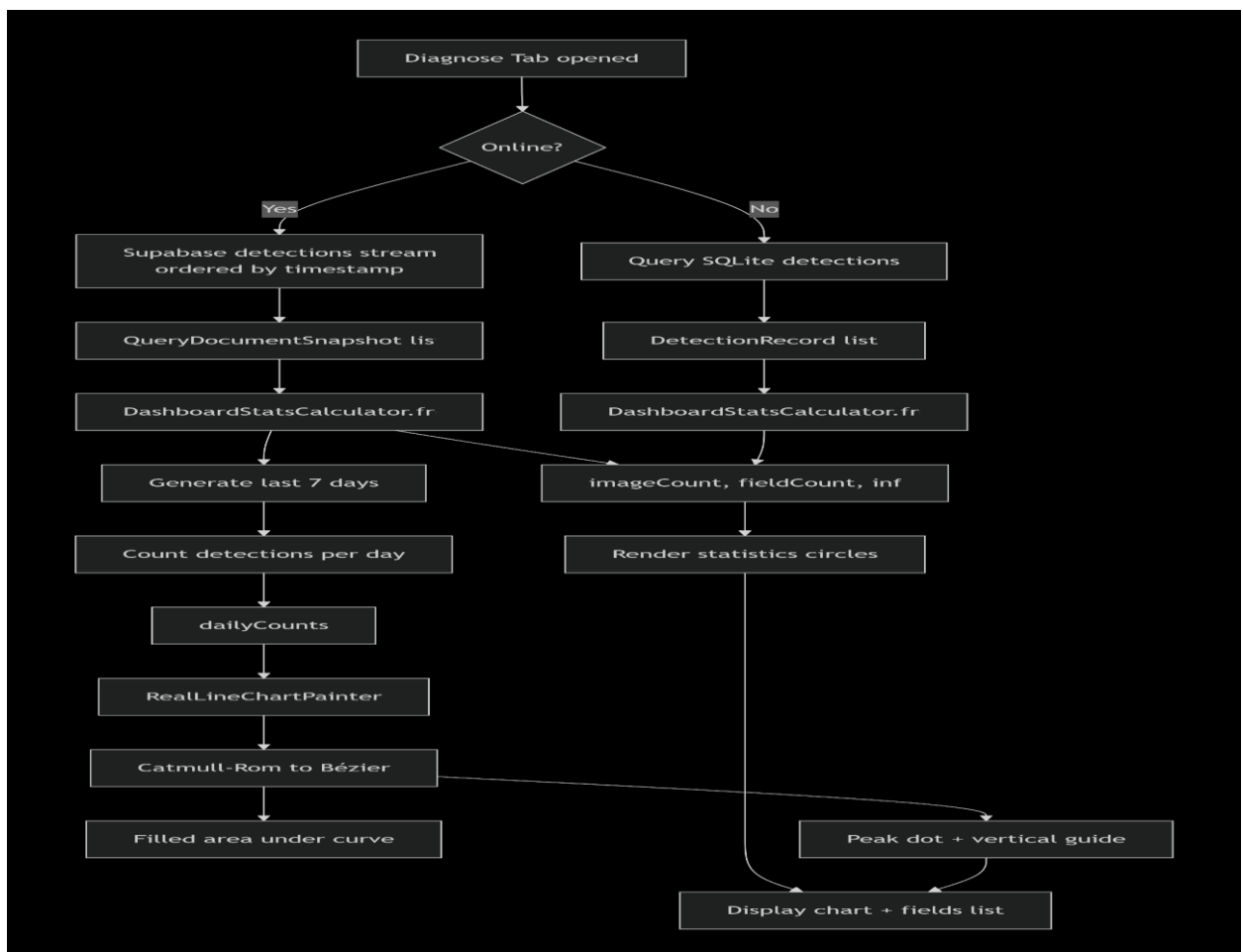


Diagram B5: Field Management Flow

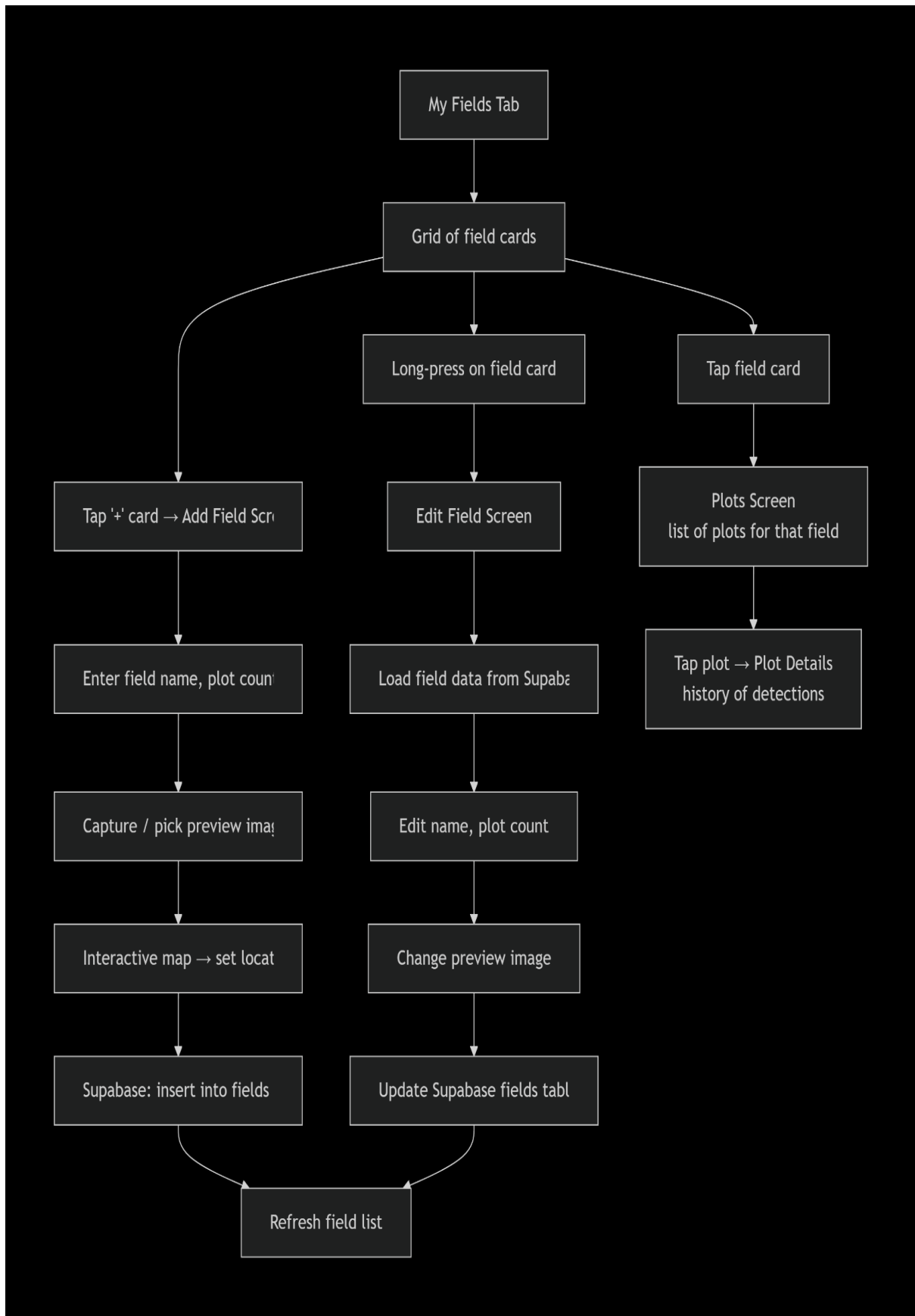


Diagram B6: Entity Relationship Diagram (Cloud + Local)

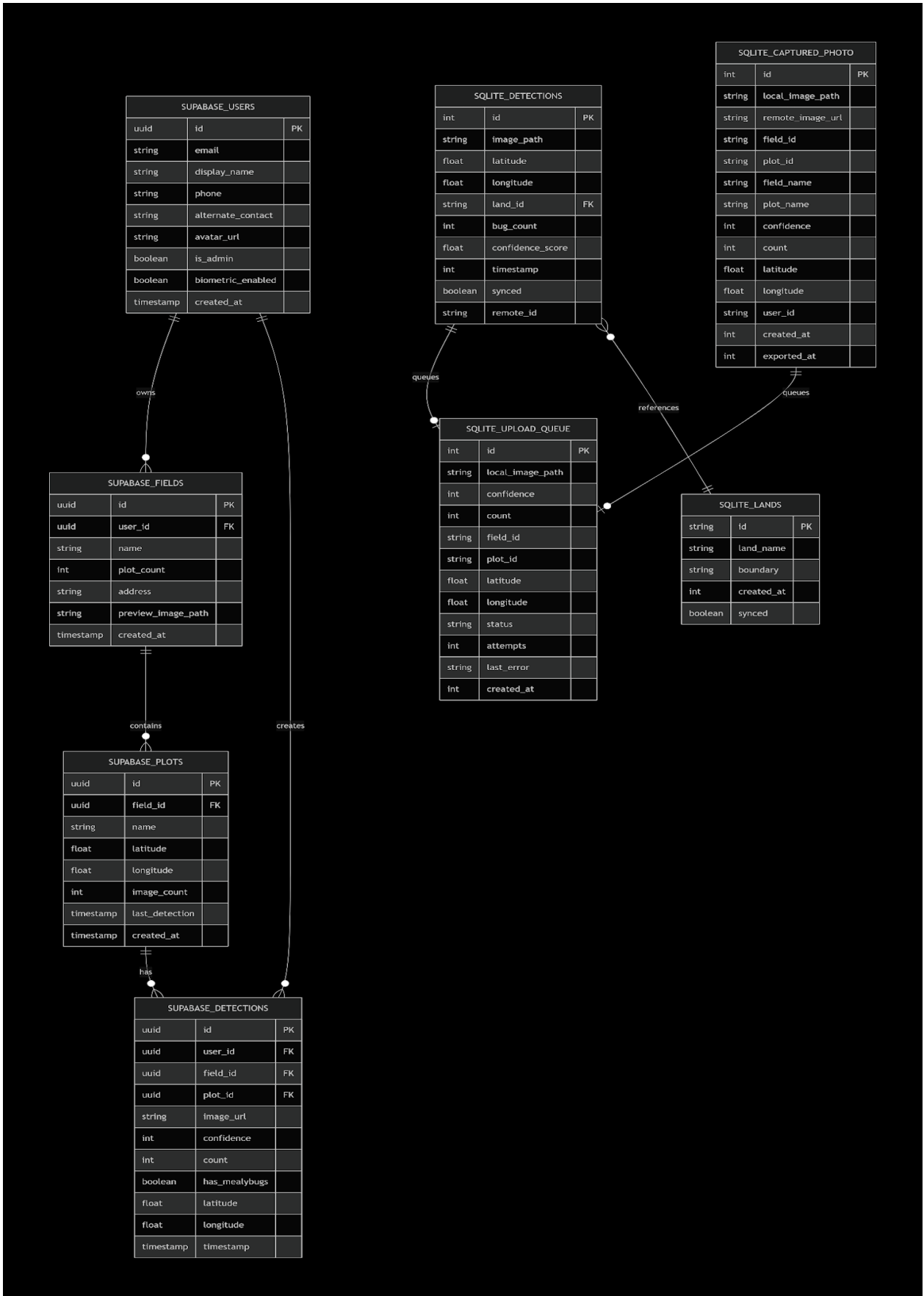


Diagram B7: Service Dependency Graph

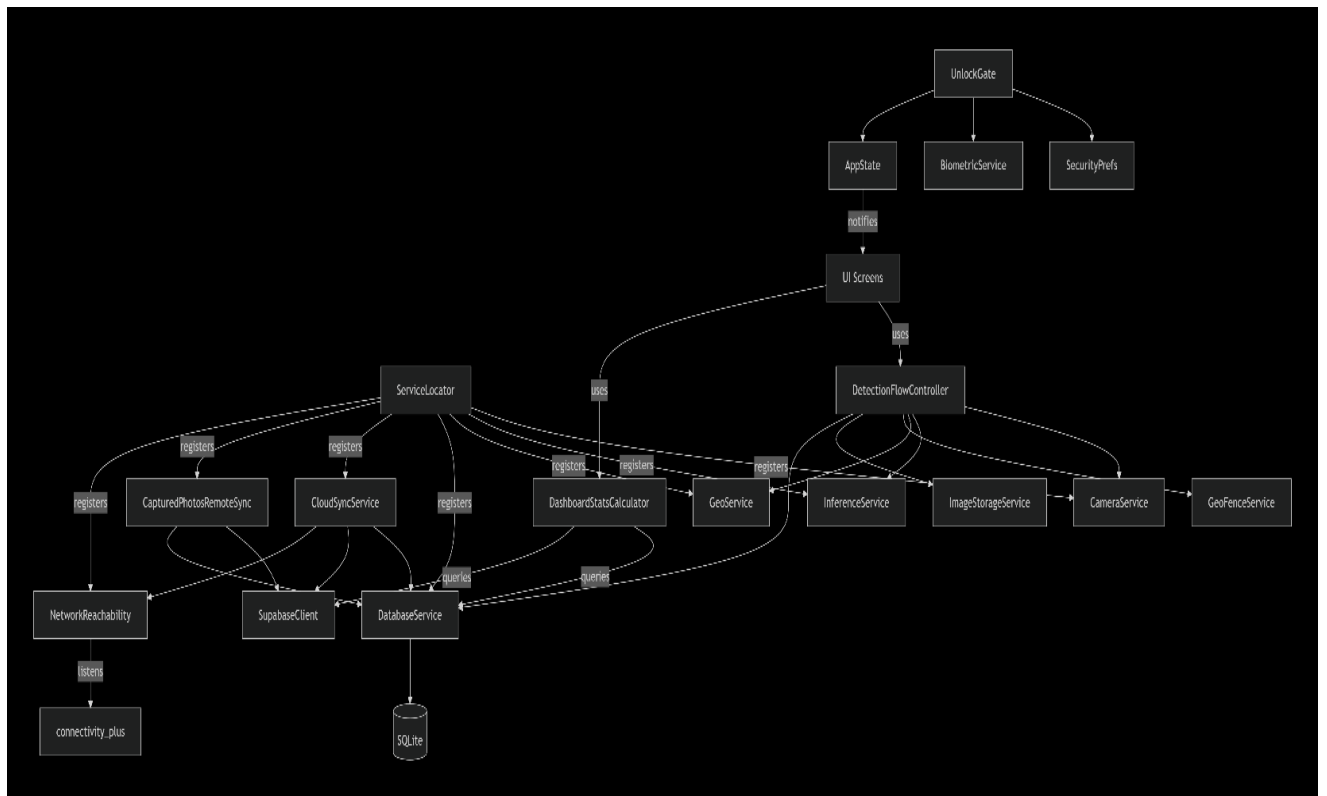


Diagram B8: Sequence Diagram – Detection Flow

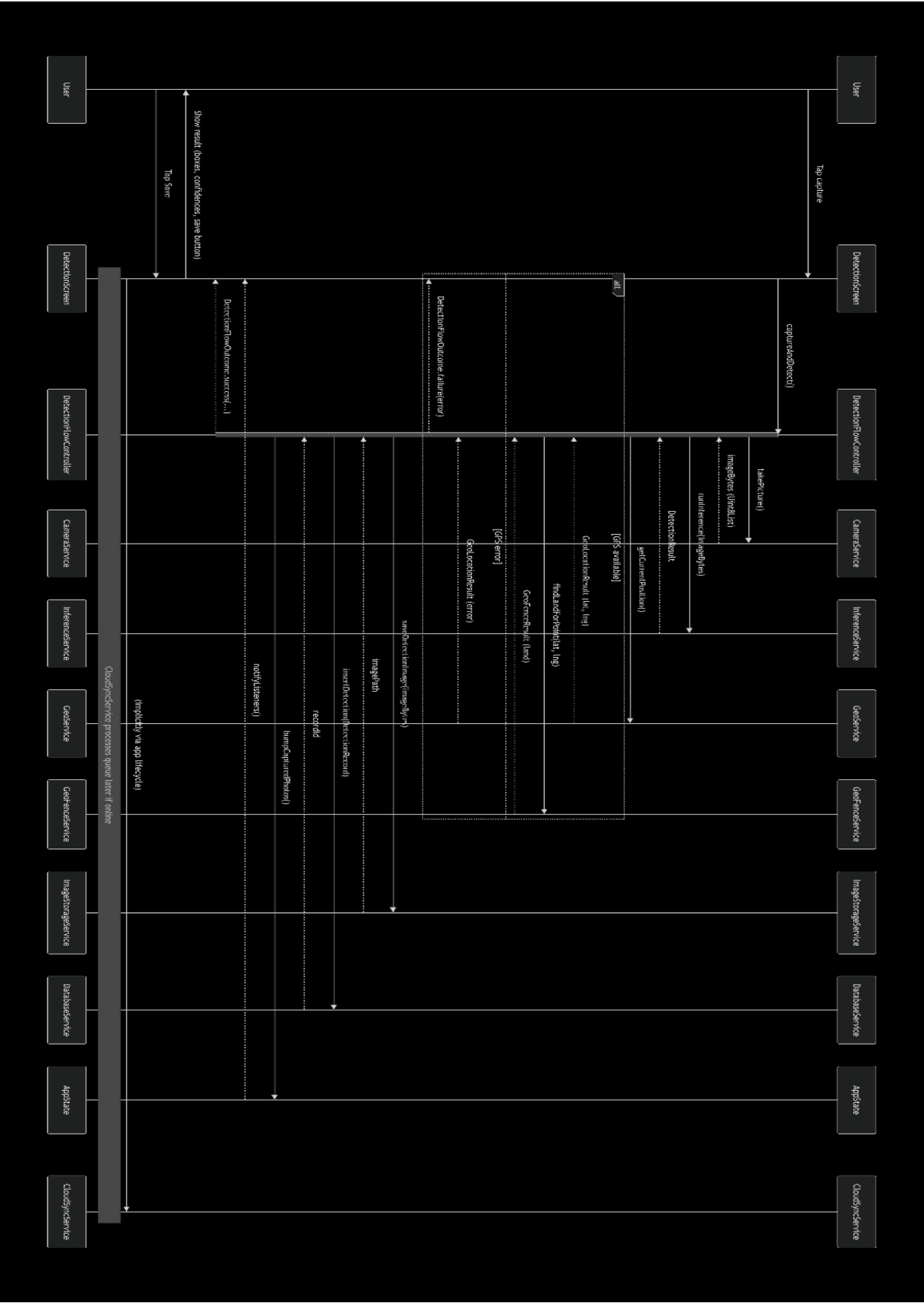


Diagram B9: Sequence Diagram – Offline Sync

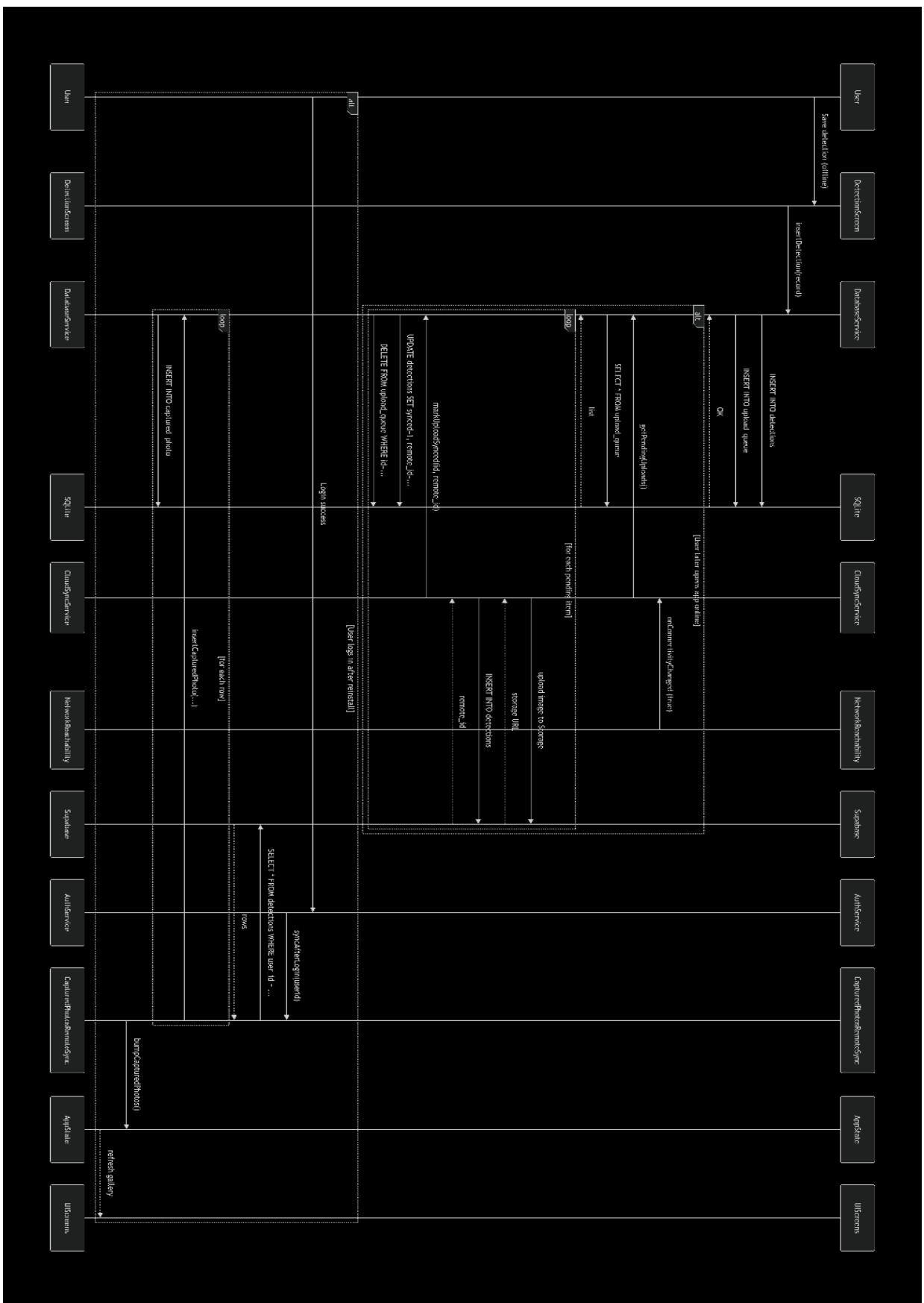


Diagram B10: Deployment Diagram

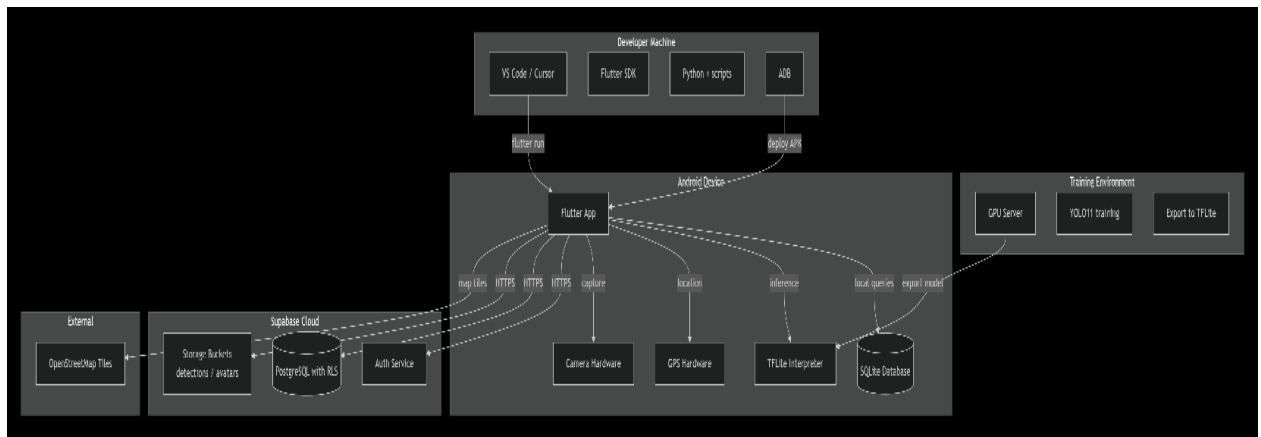


Diagram B11: Component Architecture Diagram

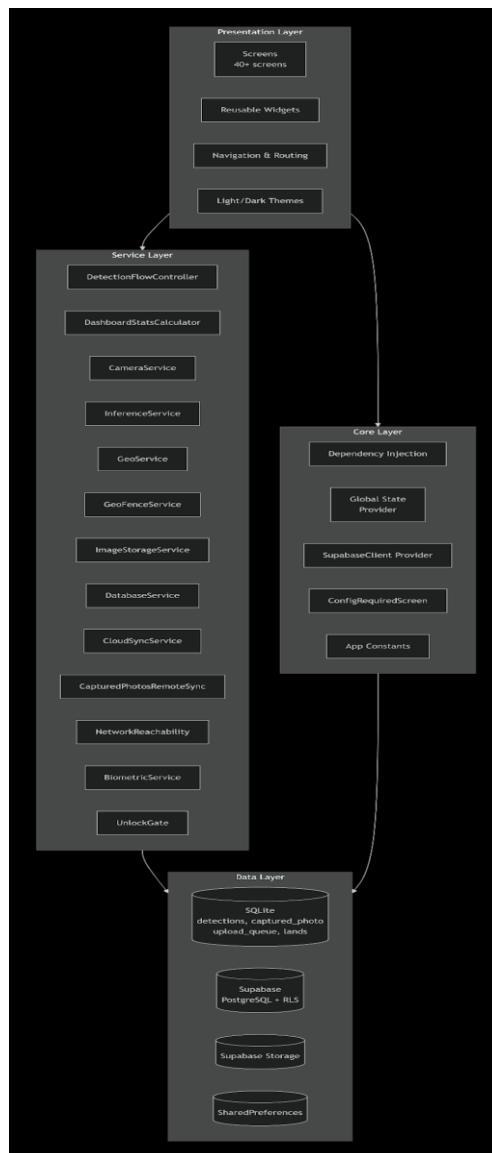


Diagram B12: Feedback Flow

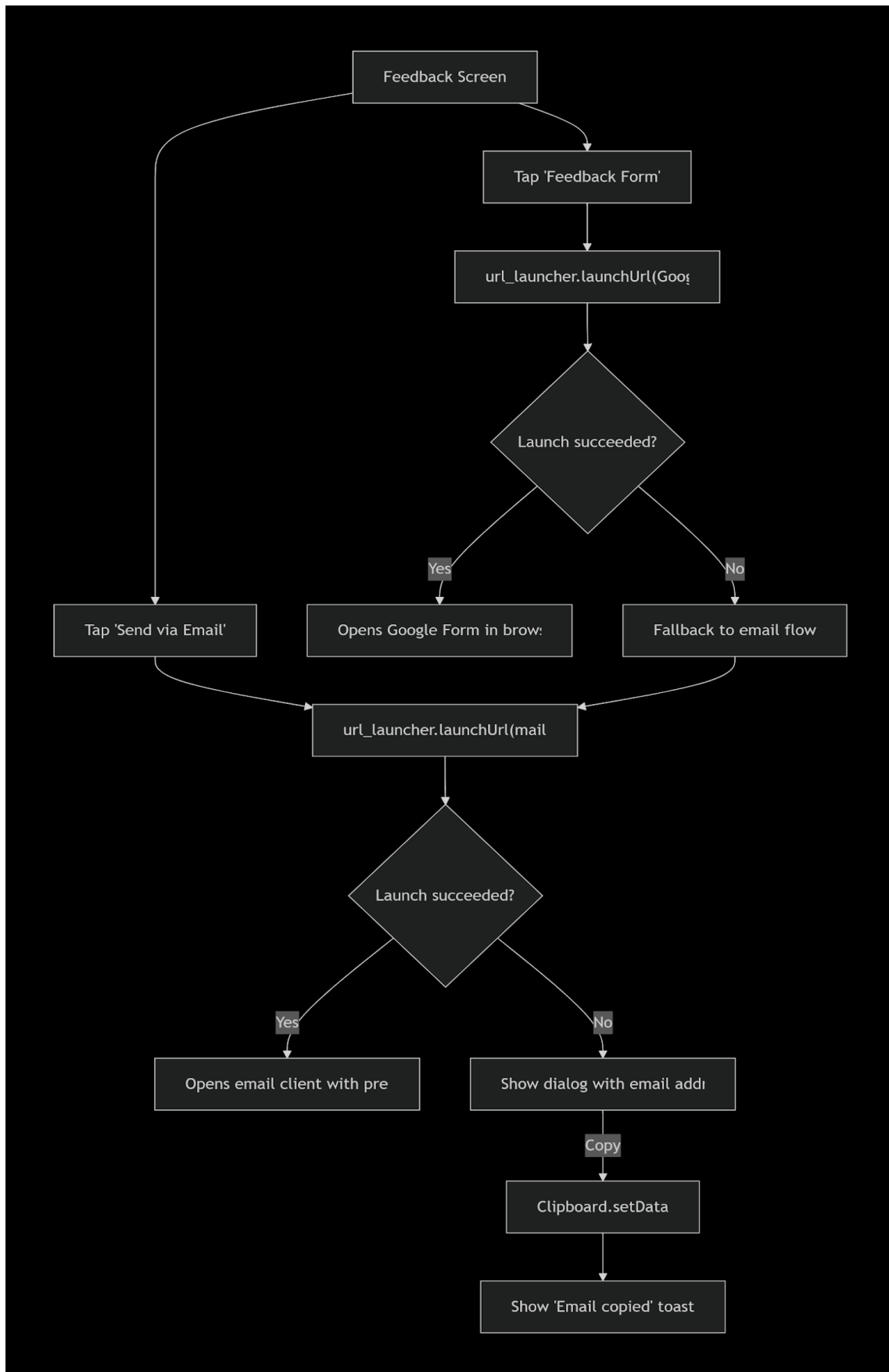
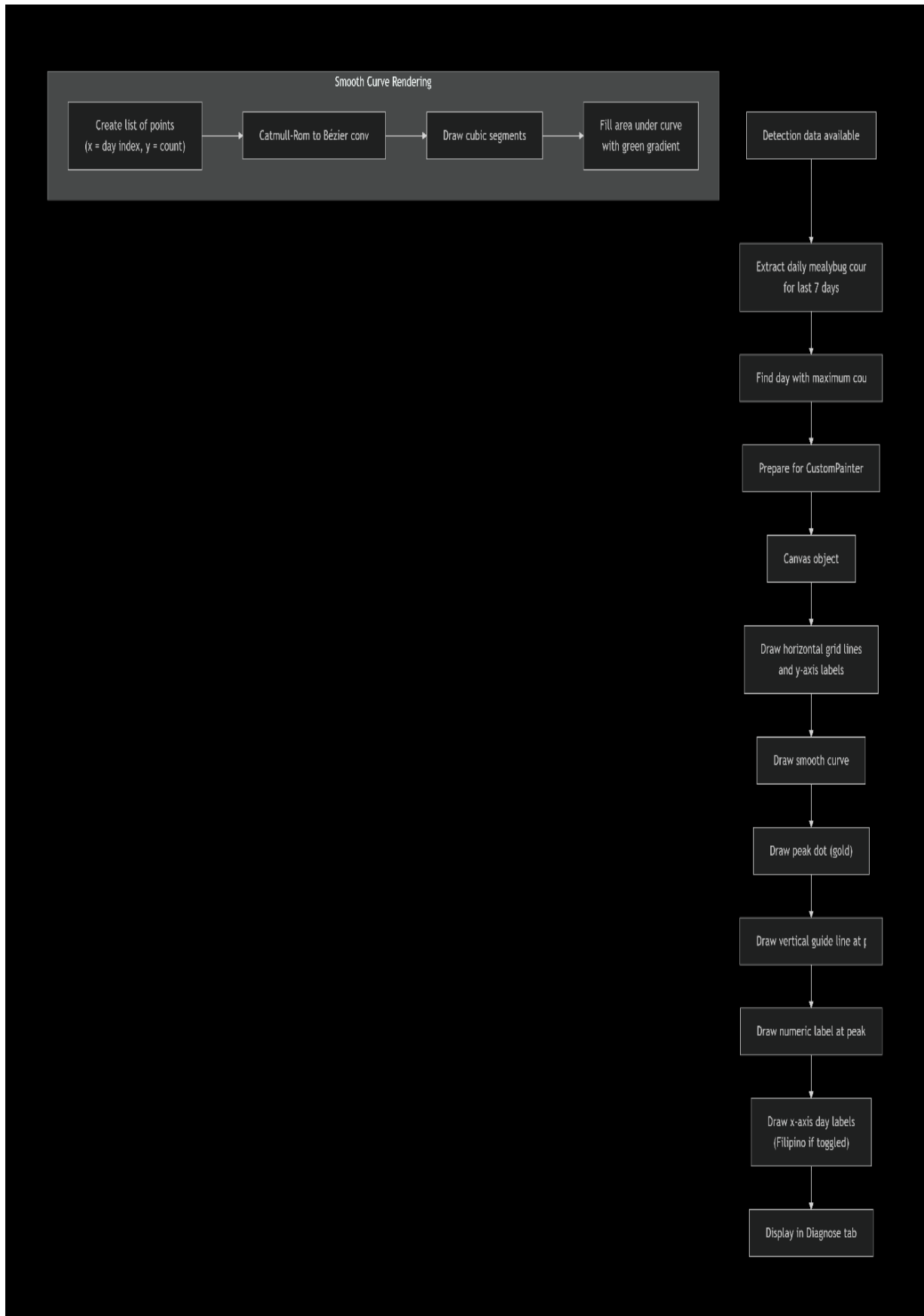


Diagram B13: Profile Management Flow



Diagram B14: Chart Rendering Flow



REFERENCES

- Bendre, M. R., Thool, R. C., & Thool, V. R. (2021). Big data in precision agriculture: Weather forecasting for future farming. *1st International Conference on Next Generation Computing Technologies*. Springer.
- Canario, D. V. P., Natsuaki, K. T., & Jahn, G. C. (2017). Pineapple mealybug wilt disease. *Plant Pathology*, 66(1), 1–15.
- Chen, J., Lin, S., He, G., & Huang, J. (2021). AI-based pest detection using deep learning models. *Computers and Electronics in Agriculture*, 187, 106287.
- Chen, L., Wang, Y., & Zhang, H. (2024). Offline-first architectures for mobile applications in resource-constrained environments. *Journal of Mobile Computing*, 12(3), 45–62.
- Esgario, J. G. M., Krohling, R. A., & Ventura, J. A. (2022). Deep learning for diagnosis of leaf diseases and pests in coffee crops. *Computers and Electronics in Agriculture*, 194, 106742.
- Hazarika, S., Saikia, M., & Borah, R. (2023). Electronic nose for mealybug infestation detection in Khasi Mandarin orange plants. *Sensors and Actuators B: Chemical*, 381, 133456.
- Hossain, M. F. (2015). World pineapple production: An overview. *African Journal of Food, Agriculture, Nutrition and Development*, 15(2), 9875–9892.
- Hussain, M. (2024). YOLO-v1 to YOLO-v8: A comprehensive review of object detection algorithms. *Digital Signal Processing*, 147, 104416.
- Li, W., Zheng, T., Yang, Z., Li, M., & Sun, C. (2022). Object recognition of crop-damaging insect pests based on YOLOv5. *Agronomy*, 12(5), 1099.

- N'Guessan, L., Koffi, K. K., & Kouadio, Y. J. (2024). Pineapple mealybug wilt-associated viruses: A comprehensive review. *Virus Research*, 339, 199270.
- Sauro, J. (2011). *A Practical Guide to the System Usability Scale: Background, Benchmarks & Best Practices*. Measuring Usability LLC.
- Sharma, P., Gupta, A., & Singh, R. (2024). Comparative analysis of YOLOv8–v11 and Faster R-CNN for multi-species weed detection. *Precision Agriculture*, 25(4), 1–22.
- Shilaskar, S., Ghatol, A., & Patil, P. (2023). Vision-based detection of mealybug infections in custard apple plants. *International Journal of Intelligent Systems and Applications in Engineering*, 11(2), 455–463.
- Soni, V., Shrivastava, L., & Jain, S. (2021). Geotagging-based mobile application for agricultural sample tracking. *International Journal of Advanced Research in Computer Science*, 12(3), 33–39.
- Terven, J. & Cordova-Esparza, D. (2023). A comprehensive review of YOLO architectures in computer vision. *Machine Learning and Knowledge Extraction*, 5(4), 1680–1716.
- Wang, X., Liu, Y., & Chen, Z. (2025). Visualization techniques for agricultural data. *Computers and Electronics in Agriculture*, 218, 108672.
- Yang, C. (2020). Remote sensing and precision agriculture technologies for crop disease detection and management. *ISPRS Journal of Photogrammetry and Remote Sensing*, 166, 1–14.
- Yulita, I. N., Ramdan, A., et al. (2023). Mobile pest detection using Convolutional Neural Networks and GoogleNet model. *ICT Express*, 9(5), 942–947.

Brooke, J. (1986). SUS: A "quick and dirty" usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & A. L. McClelland (Eds.), *Usability Evaluation in Industry* (pp. 189-194). Taylor & Francis.

Document Version: v4

Last Updated: May 12, 2026

Status: Final (Supabase-based, offline-first architecture, YOLO26n TFLite detector, severity map, account-linked gallery)